

Documentación Técnica — Lab NDR: Suricata + Zeek + Grafana/Loki



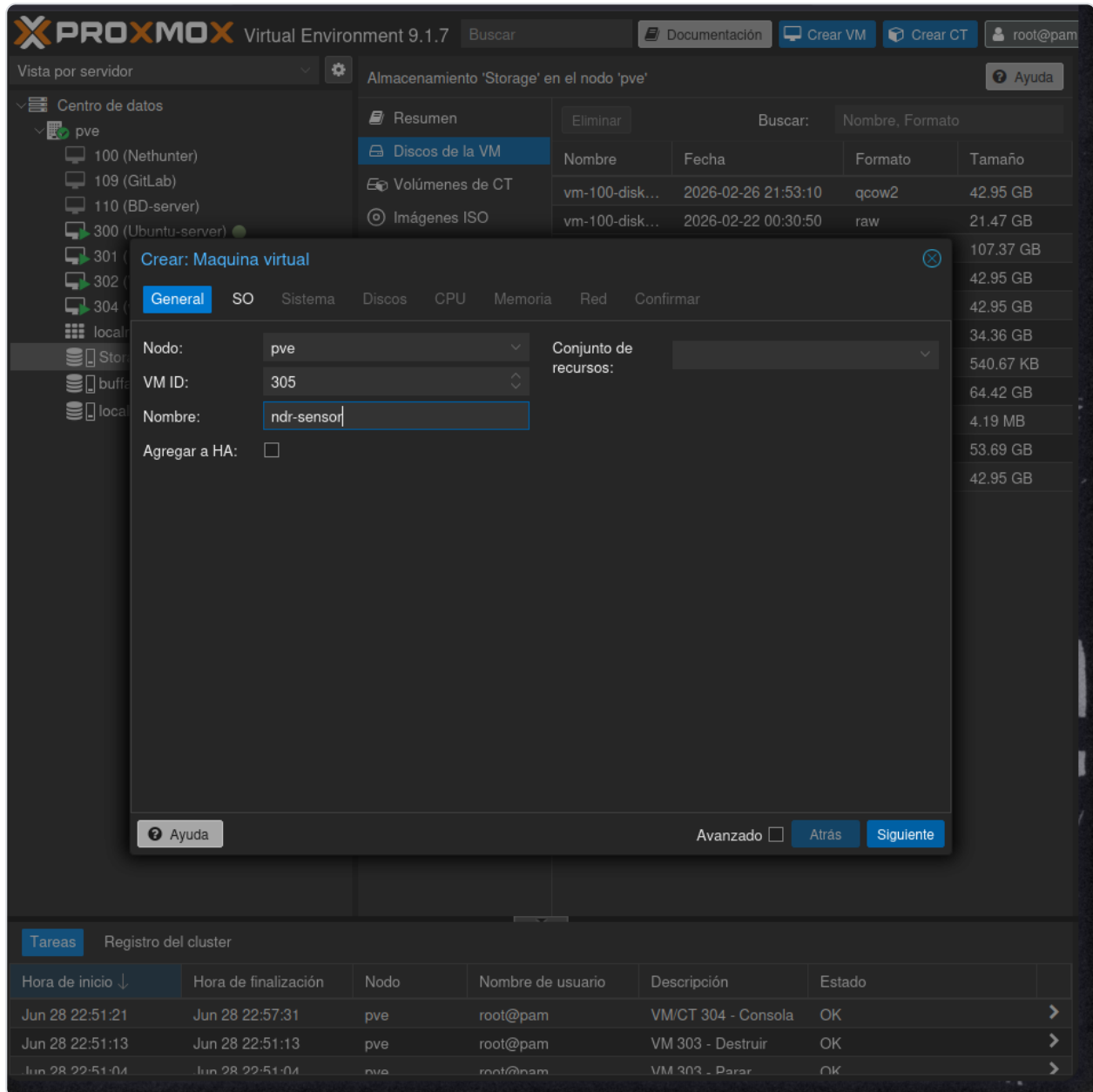
Índice

1. Creación de la máquina virtual en Proxmox
2. 1.1 General — nombre e ID de VM
3. 1.2 SO — imagen ISO y sistema operativo
4. 1.3 Sistema — controlador y firmware
5. 1.4 Discos — tamaño y formato
6. 1.5 CPU — núcleos y tipo de procesador
7. 1.6 Memoria — RAM asignada
8. 1.7 Red — primera interfaz de red
9. 1.8 Confirmación final de la VM
10. 1.9 Agregar interfaz de captura
11. Instalación de Ubuntu Server 24.04
12. 2.1 Selección de idioma
13. 2.2 Configuración de almacenamiento
14. 2.3 Configuración de perfil de usuario
15. 2.4 Configuración de SSH
16. 2.5 Progreso de instalación
17. Configuración de red y modo promiscuo
18. 3.1 Agregar segunda NIC de captura en Proxmox
19. 3.2 Configuración de red con Netplan
20. 3.3 Aplicación de Netplan y verificación
21. 3.4 Estado completo de interfaces de red
22. 3.5 Activar modo promiscuo en ens20
23. 3.6 Servicio systemd para persistencia del modo promiscuo
24. 3.7 Confirmación del servicio promiscuo activo
25. Instalación y configuración de Suricata
26. 4.1 Añadir repositorio oficial OISF
27. 4.2 Instalación del paquete Suricata
28. 4.3 Verificación de versión instalada
29. 4.4 Configuración de HOME_NET
30. 4.5 Configuración de interfaz af-packet
31. 4.6 Descarga de reglas Emerging Threats
32. 4.7 Revisión y test de configuración
33. 4.8 Estado del servicio Suricata
34. 4.9 Verificación de logs eve.json
35. Despliegue de Zeek con Docker
36. 5.1 Instalación de Docker
37. 5.2 Docker habilitado y en ejecución

- 38. 5.3 [Comando de lanzamiento del contenedor Zeek](#)
 - 39. 5.4 [Verificación del contenedor y escucha en ens20](#)
 - 40. 5.5 [Logs de conexiones capturadas](#)
 - 41. 5.6 [Filtrado de tráfico local](#)
 - 42. [Stack de visualización: Grafana + Loki + Promtail](#)
 - 43. 6.1 [Contenido del docker-compose.yml](#)
 - 44. 6.2 [Configuración de Promtail](#)
 - 45. 6.3 [Descarga e inicio del stack](#)
 - 46. 6.4 [Servicios corriendo](#)
 - 47. 6.5 [Interfaz de conexiones en Grafana](#)
 - 48. 6.6 [Configuración de la fuente de datos Loki](#)
 - 49. 6.7 [Pantalla de creación de dashboards](#)
 - 50. 6.8 [Página de inicio de Grafana](#)
 - 51. 6.9 [Editor de panel — query de logs Suricata](#)
 - 52. 6.10 [Panel con logs de Suricata en tiempo real](#)
 - 53. 6.11 [Editor de panel — query de logs Zeek](#)
 - 54. [Evidencias de detección](#)
 - 55. 7.1 [Verificación de alertas con jq sobre eve.json](#)
-

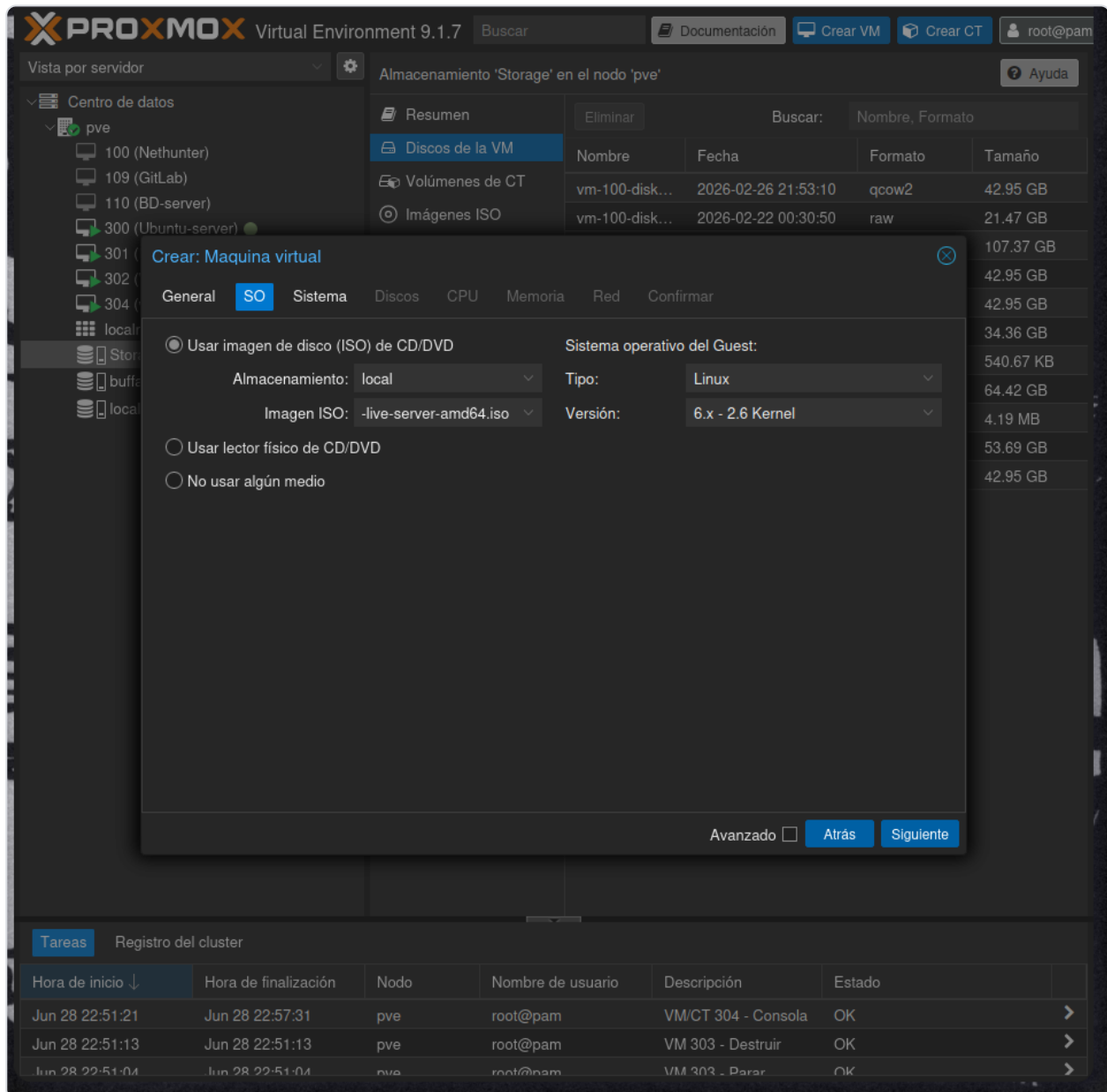
1. Creación de la máquina virtual en Proxmox

1.1 General — nombre e ID de VM



En el asistente de creación de Proxmox VE 9.1.7 se asigna el **VM ID 305** y el nombre **ndr-sensor**. El nodo de destino es **pve**. En el panel lateral izquierdo se puede ver el inventario de VMs del clúster, entre ellas **300 (Ubuntu-server)**, **301 (Pfsense-firewall)**, **302 (WS-AD)** y **304 (wazuh)**, lo que confirma que esta VM se integra en un entorno de laboratorio de ciberseguridad más amplio. El campo "Conjunto de recursos" queda vacío al no usarse Resource Pools.

1.2 SO — imagen ISO y sistema operativo



Se selecciona la imagen **ubuntu-24.04.4-live-server-amd64.iso** almacenada en el almacenamiento local del nodo. El tipo de SO se configura como **Linux** con versión de kernel **6.x - 2.6 Kernel**, que es la opción correcta para Ubuntu 24.04 LTS. No se utiliza lector de CD/DVD físico, sino la ISO montada como dispositivo IDE2.

1.3 Sistema — controlador y firmware

The screenshot displays the Proxmox Virtual Environment (VE) 9.1.7 web interface. The main menu on the left shows a list of VMs under the 'pve' node. The central panel shows the 'Almacenamiento "Storage" en el nodo "pve"' section with a table of VM disks. The 'Crear: Maquina virtual' dialog box is open, showing the 'Sistema' tab. The configuration options for the system are as follows:

General	SO	Sistema	Discos	CPU	Memoria	Red	Confirmar
Tarjeta gráfica:	Por defecto	Controlador SCSI:	VirtIO SCSI single				
Maquina:	Por defecto (i440fx)	Qemu Agent:	☐				
Firmware		Agregar TPM:	☐				
BIOS:	Por defecto (SeaBIOS)						

At the bottom of the dialog box, there are buttons for 'Ayuda', 'Avanzado', 'Atrás', and 'Siguiente'.

The background shows a table of cluster tasks:

Hora de inicio ↓	Hora de finalización	Nodo	Nombre de usuario	Descripción	Estado
Jun 28 22:51:21	Jun 28 22:57:31	pve	root@pam	VM/CT 304 - Consola	OK
Jun 28 22:51:13	Jun 28 22:51:13	pve	root@pam	VM 303 - Destruir	OK
Jun 28 22:51:04	Jun 28 22:51:04	pve	root@pam	VM 303 - Parar	OK

Se configura:

- **Controlador SCSI:** VirtIO SCSI single — ofrece el mejor rendimiento de disco en entornos KVM/QEMU al usar el driver paravirtualizado de VirtIO.
- **Máquina:** Por defecto (i440fx) — chipset estable y ampliamente compatible con Linux.
- **BIOS:** Por defecto (SeaBIOS) — firmware estándar, sin necesidad de UEFI para esta VM.
- **Qemu Agent** y **TPM** se dejan desactivados, lo que es adecuado para un servidor NDR sin requisitos de seguridad de arranque medido.

1.4 Discos — tamaño y formato

The screenshot displays the Proxmox VE 9.1.7 web interface. The main window is titled 'Crear: Maquina virtual' (Create: Virtual Machine). The 'Discos' (Disks) tab is selected, showing the configuration for a new disk. The disk is named 'scsi0' and is located in the 'Storage' pool. The size is set to 40 GiB, and the format is 'Imagen de disco QEI'. The 'IO thread' is checked, and the 'Cache' is set to 'Por defecto (No hay)'. The 'Advanced' checkbox is unchecked. The background shows a list of existing VMs and a cluster task log.

Nombre	Fecha	Formato	Tamaño
vm-100-disk...	2026-02-26 21:53:10	qcow2	42.95 GB
vm-100-disk...	2026-02-22 00:30:50	raw	21.47 GB
			107.37 GB
			42.95 GB
			42.95 GB
			34.36 GB
			540.67 KB
			64.42 GB
			4.19 MB
			53.69 GB
			42.95 GB

Hora de inicio ↓	Hora de finalización	Nodo	Nombre de usuario	Descripción	Estado
Jun 28 22:51:21	Jun 28 22:57:31	pve	root@pam	VM/CT 304 - Consola	OK
Jun 28 22:51:13	Jun 28 22:51:13	pve	root@pam	VM 303 - Destruir	OK
Jun 28 22:51:04	Jun 28 22:51:04	pve	root@pam	VM 303 - Parar	OK

Se asigna un único disco en el bus **SCSI0** con las siguientes características:

- **Controlador SCSI:** VirtIO SCSI single
- **Almacenamiento:** pool **Storage**
- **Tamaño:** 40 GB — suficiente para el SO, Suricata, Docker y los volúmenes de logs
- **Formato:** Imagen de disco QEMU (qcow2) — permite snapshots y thin provisioning
- **IO Thread:** activado — mejora el rendimiento de I/O al dedicar un hilo exclusivo al disco
- **Caché:** Por defecto (No hay) — comportamiento seguro sin pérdida de datos ante fallos

1.5 CPU — núcleos y tipo de procesador

The screenshot displays the Proxmox Virtual Environment 9.1.7 interface. A modal window titled 'Crear: Máquina virtual' is open, showing the 'CPU' configuration tab. The 'Zócalos' (Sockets) is set to 1, and 'Núcleos' (Cores) is set to 4. The 'Tipo' (Type) is set to 'x86-64-v2-AES'. The 'Total de Núcleos' (Total Cores) is 4. The background shows a list of VMs and a table of cluster tasks.

Nombre	Fecha	Formato	Tamaño
vm-100-disk...	2026-02-26 21:53:10	qcow2	42.95 GB
vm-100-disk...	2026-02-22 00:30:50	raw	21.47 GB

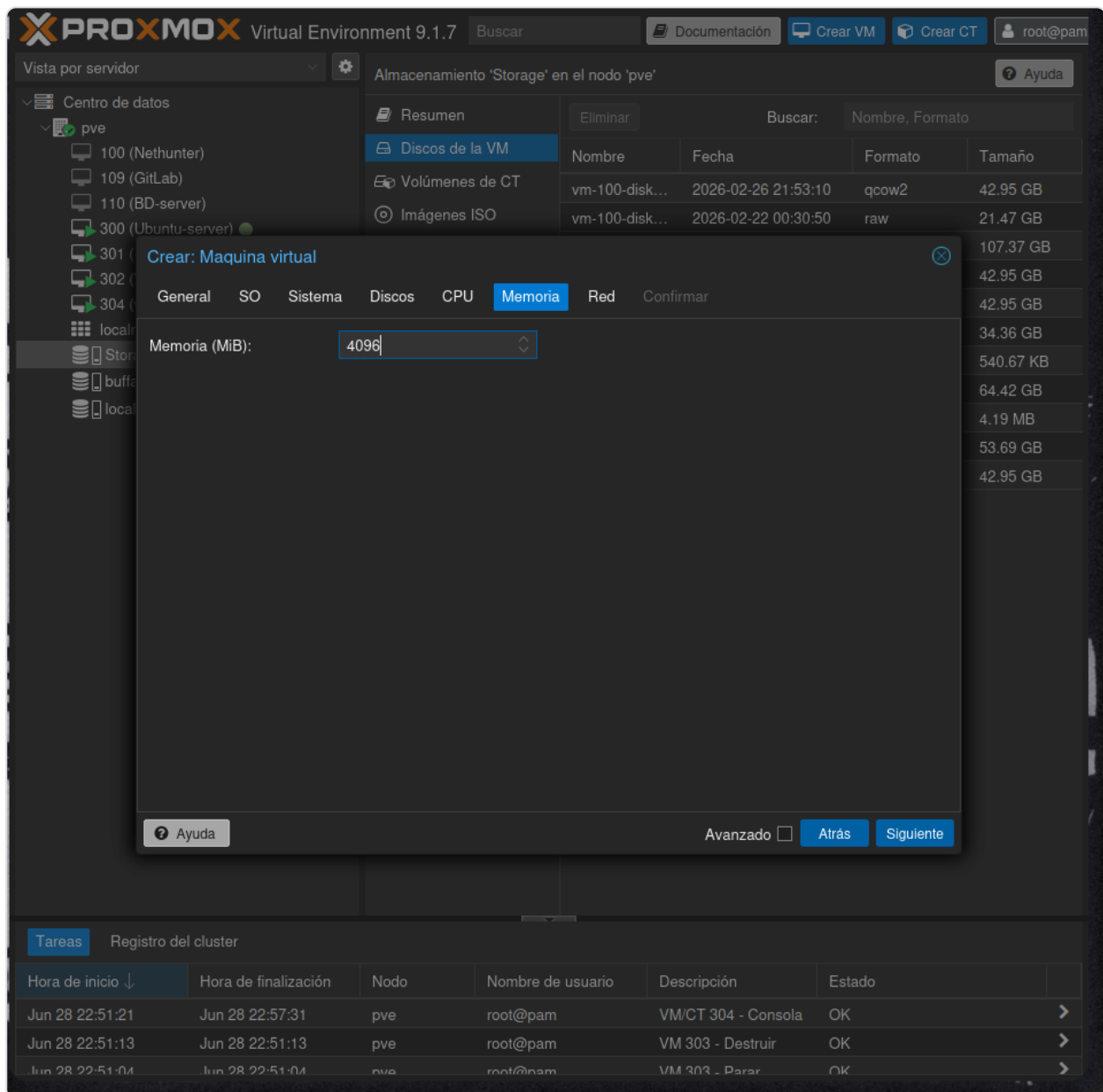
Horario de inicio ↓	Horario de finalización	Nodo	Nombre de usuario	Descripción	Estado
Jun 28 22:51:21	Jun 28 22:57:31	pve	root@pam	VM/CT 304 - Consola	OK
Jun 28 22:51:13	Jun 28 22:51:13	pve	root@pam	VM 303 - Destruir	OK

La CPU se configura con:

- **Zócalos:** 1 socket físico simulado
- **Núcleos:** 4 cores — necesarios para el rendimiento multihilo de Suricata (que escala por núcleo con af-packet)
- **Tipo:** x86-64-v2-AES — incluye soporte nativo de AES-NI y las extensiones del nivel v2 del ISA x86-64, optimizando el procesamiento de tráfico TLS/SSL cifrado que Suricata debe inspeccionar

Total de núcleos: 4 (1 socket × 4 cores)

1.6 Memoria — RAM asignada



Se asignan **4096 MiB (4 GB)** de memoria RAM. Esta cantidad es suficiente para ejecutar simultáneamente:

- Ubuntu Server 24.04 (sistema base)
- Suricata 8.0.5 con 50.000+ reglas cargadas (~213 MB según el status)
- Zeek en contenedor Docker
- Stack NDR (Grafana + Loki + Promtail) en Docker Compose

No se activa el ballooning de memoria, lo que garantiza que los 4 GB están siempre reservados para esta VM.

1.7 Red — primera interfaz de red

The screenshot displays the Proxmox VE 9.1.7 web interface. A modal window titled 'Crear: Máquina virtual' is open, showing the 'Red' (Network) tab. The configuration for the network interface is as follows:

- Sin dispositivo de red:** ☐
- Puente:** vmbr0
- Modelo:** VirtIO (paravirtualizado)
- Etiqueta VLAN:** Ninguna VLAN
- Dirección MAC:** auto
- Cortafuego:** ☒

The background interface shows a list of VMs on the left and a table of cluster tasks at the bottom.

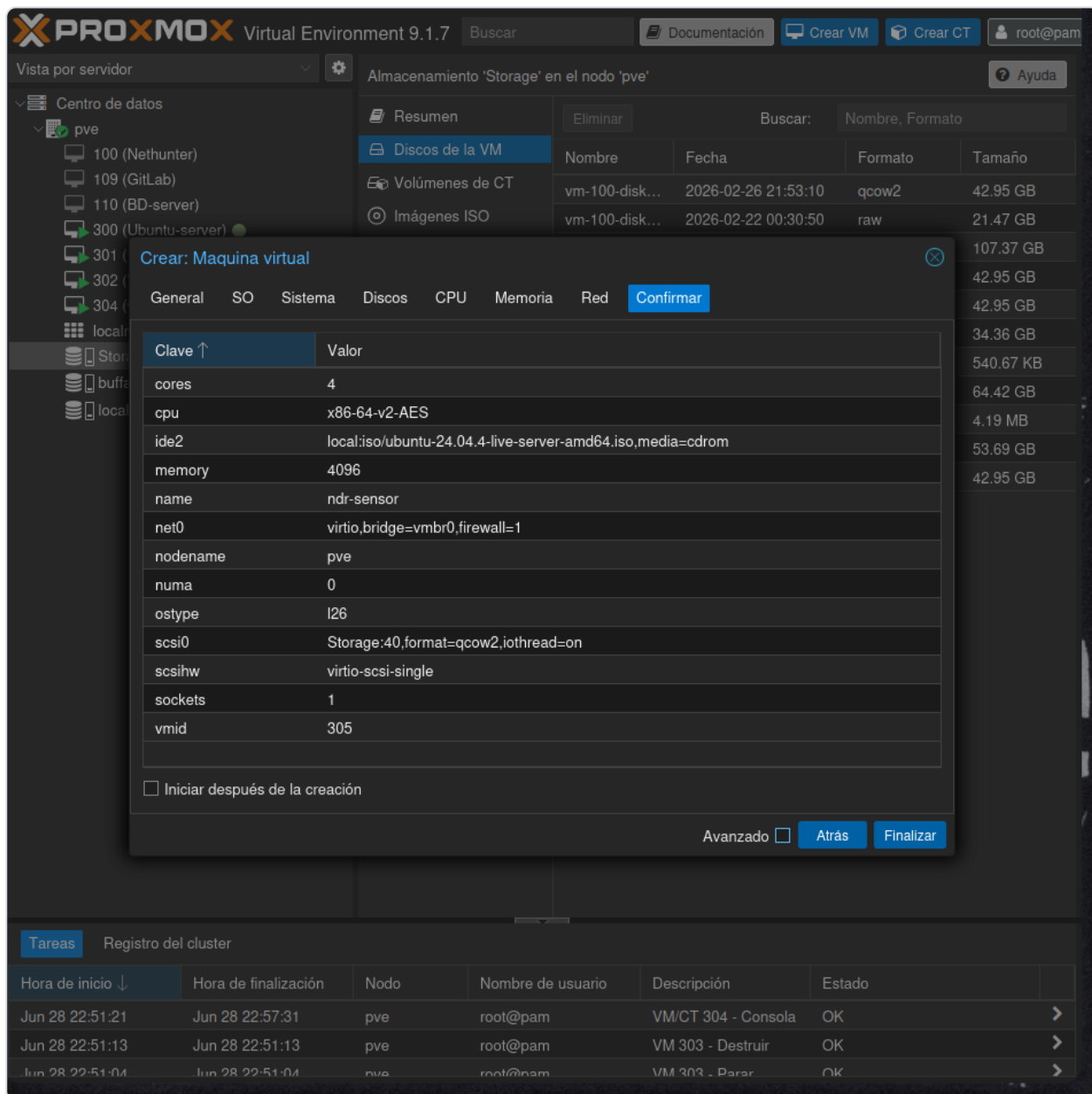
Hora de inicio ↓	Hora de finalización	Nodo	Nombre de usuario	Descripción	Estado
Jun 28 22:51:21	Jun 28 22:57:31	pve	root@pam	VM/CT 304 - Consola	OK
Jun 28 22:51:13	Jun 28 22:51:13	pve	root@pam	VM 303 - Destruir	OK
Jun 28 22:51:04	Jun 28 22:51:04	pve	root@pam	VM 303 - Parar	OK

La primera interfaz de red se configura con:

- **Puente:** vmbr0 — bridge de gestión/LAN principal del hipervisor
- **Modelo:** VirtIO (paravirtualizado) — driver de red de máximo rendimiento en KVM
- **Etiqueta VLAN:** Ninguna — tráfico sin segmentación VLAN en este bridge
- **Cortafuego:** desactivado — la inspección de tráfico la realiza Suricata, no el firewall de Proxmox

Esta interfaz se corresponde con **ens18** dentro de Ubuntu y recibirá su IP por DHCP (192.168.1.228/24). Es la interfaz de administración del sensor.

1.8 Confirmación final de la VM



Pantalla de resumen final antes de crear la VM. Se puede ver la configuración completa en formato clave-valor:

Clave	Valor
cores	4
cpu	x86-64-v2-AES
ide2	local:iso/ubuntu-24.04.4-live-server-amd64.iso,media=cdrom
memory	4096
name	ndr-sensor
net0	virtio, bridge=vbr0, firewall=1
numa	0

Clave	Valor
ostype	l26
scsi0	Storage:40, format=qcow2, iothread=on
scsihw	virtio-scsi-single
sockets	1
vmid	305

La opción **"Iniciar después de la creación"** está desmarcada, lo que permite agregar interfaces de red adicionales antes del primer arranque.

1.9 Agregar interfaz de captura

The screenshot shows the Proxmox Virtual Environment 9.1.7 interface. The main window displays the configuration for a virtual machine named "Maquina virtual 305 (ndr-sensor) en el nodo pve". The "Hardware" tab is selected, showing various components like Memory (4.00 GiB), Processors (4), BIOS, Screen, Machine, SCSI Controller, CD/DVD, Hard Disk, and Network Device. A dialog box titled "Agregar: Dispositivo de red" is open, allowing the user to add a new network interface. The dialog includes fields for Bridge (vmbr1), Model (VirtIO (paravirtualizado)), VLAN Tag (Ninguna VLAN), MAC Address (auto), and Firewall (checked). There are buttons for "Ayuda", "Avanzado", and "Agregar".

Agregar: Dispositivo de red

Puente: Modelo:

Etiqueta VLAN: Dirección MAC:

Cortafuego: ☒

☐

Tareas Registro del cluster

Hora de inicio ↓	Hora de finalización	Nodo	Nombre de usuario	Descripción	Estado
Jun 28 23:06:58	Jun 28 23:07:15	pve	root@pam	VM 305 - Crear	OK

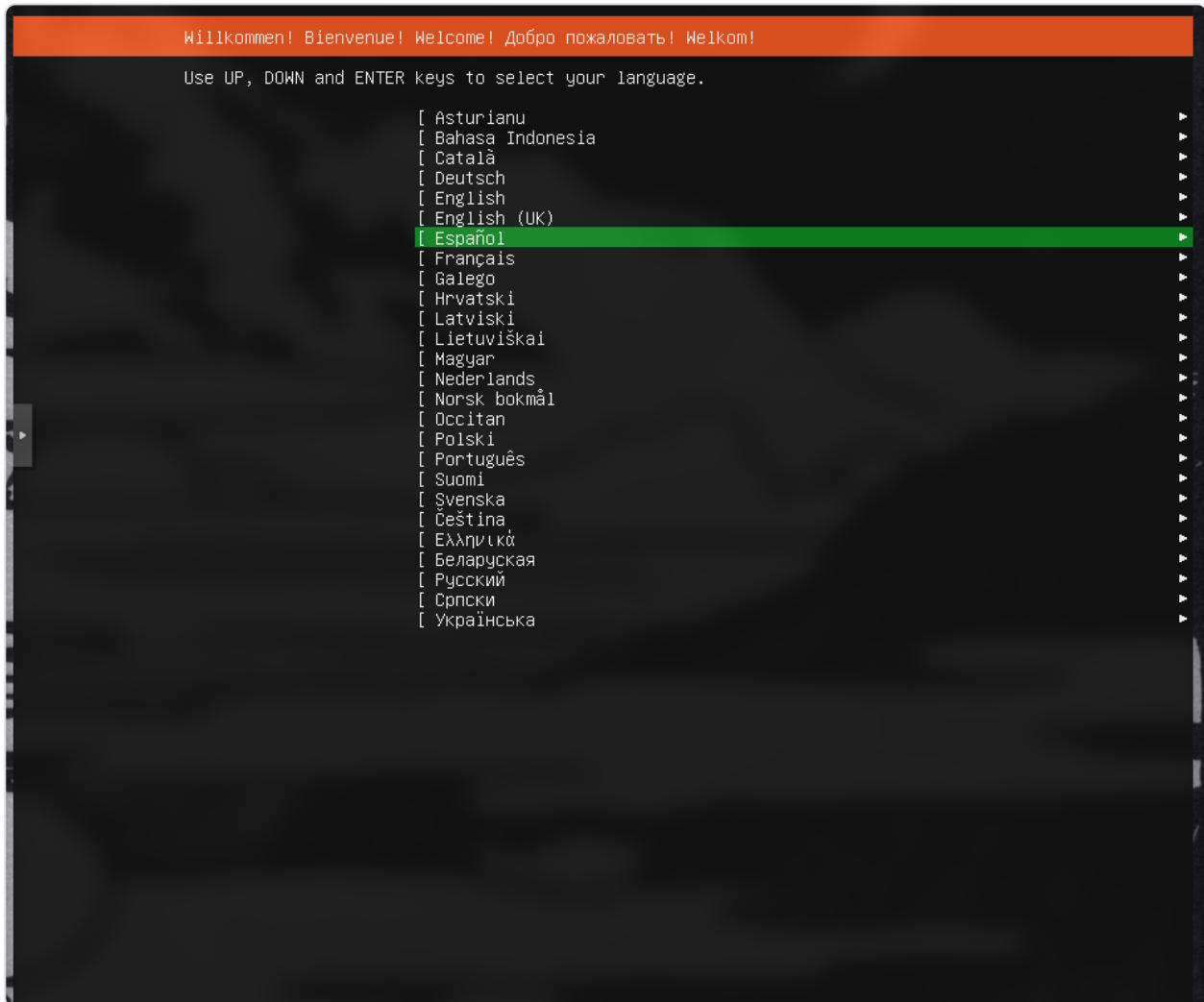
Una vez creada la VM, desde el menú **Hardware** → **Agregar** → **Dispositivo de red** se añade la segunda NIC:

- **Puente:** vmbr1 — bridge de la red interna de monitorización (192.168.100.0/24)
- **Modelo:** VirtIO (paravirtualizado)
- **Cortafuego:** activado

Esta operación se repite para agregar una tercera NIC (también en vmbr1) que será la interfaz de captura **ens20** en modo promiscuo, sin IP asignada. Tener dos NICs en el mismo bridge pero con roles distintos (ens19 con IP para comunicación y ens20 sin IP solo para captura) es una práctica habitual en sensores NDR para evitar que el tráfico de gestión contamine las capturas.

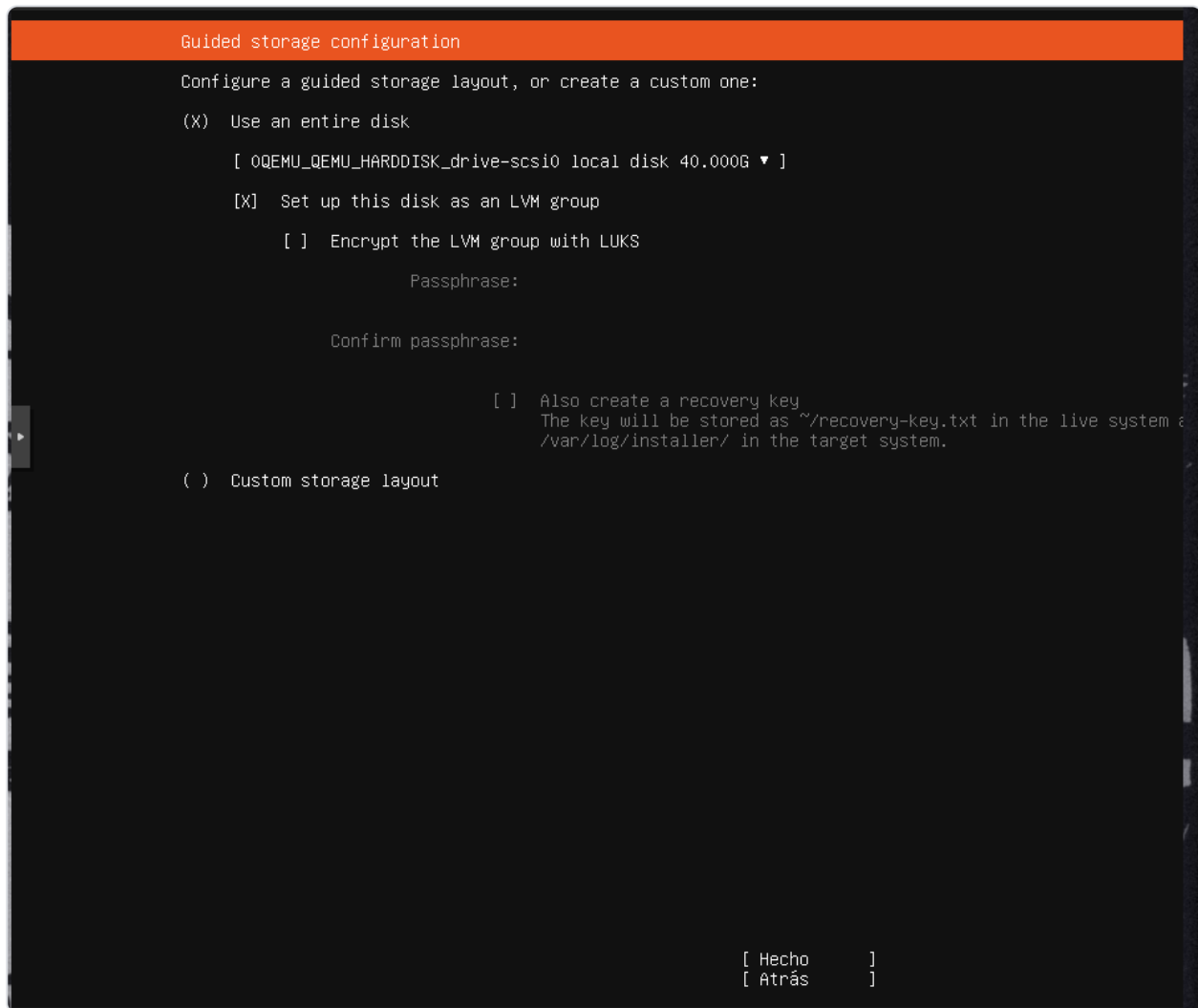
2. Instalación de Ubuntu Server 24.04

2.1 Selección de idioma



Primera pantalla del instalador Subiquity de Ubuntu 24.04.4 LTS. Se selecciona **Español** como idioma del sistema. Esta elección afecta al idioma de los mensajes del instalador y del sistema base, pero no tiene impacto en el funcionamiento de Suricata ni Zeek, que usan configuraciones en inglés de forma independiente al locale del sistema.

2.2 Configuración de almacenamiento



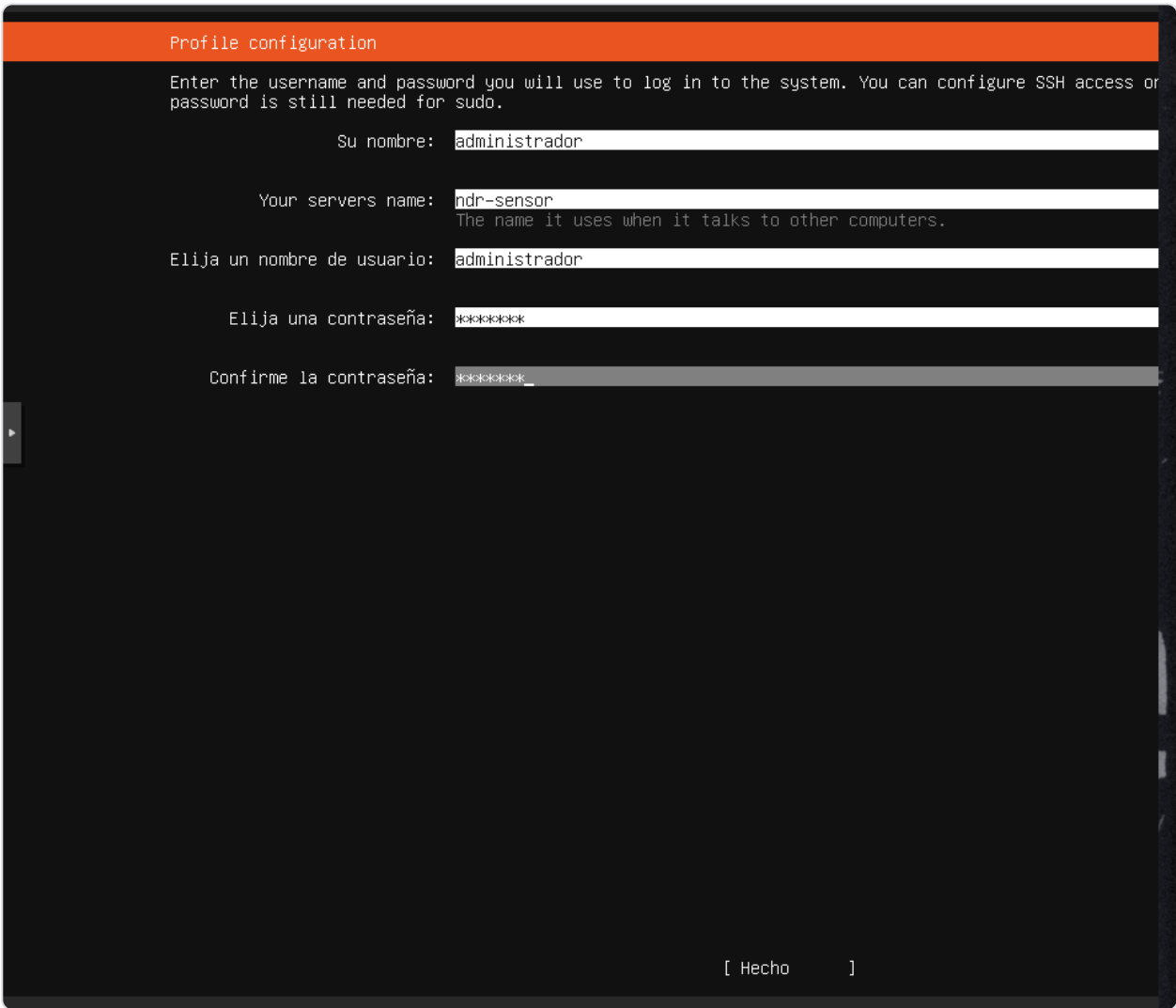
Se elige la opción **"Usar un disco completo"** sobre el dispositivo `QEMU_QEMU_HARDDISK_drive-scsi0` de 40 GB, que corresponde al disco qcow2 creado en Proxmox.

Opciones seleccionadas:

- **Configurar este disco como un grupo LVM** (activado) — permite redimensionar volúmenes lógicos en el futuro sin reparticionado
- **Cifrar el grupo LVM con LUKS** (desactivado) — no se cifra el disco ya que el sensor opera en un entorno de laboratorio controlado y el cifrado añadiría overhead en I/O de logs

El esquema LVM resultante crea automáticamente las particiones de arranque y el volumen raíz.

2.3 Configuración de perfil de usuario

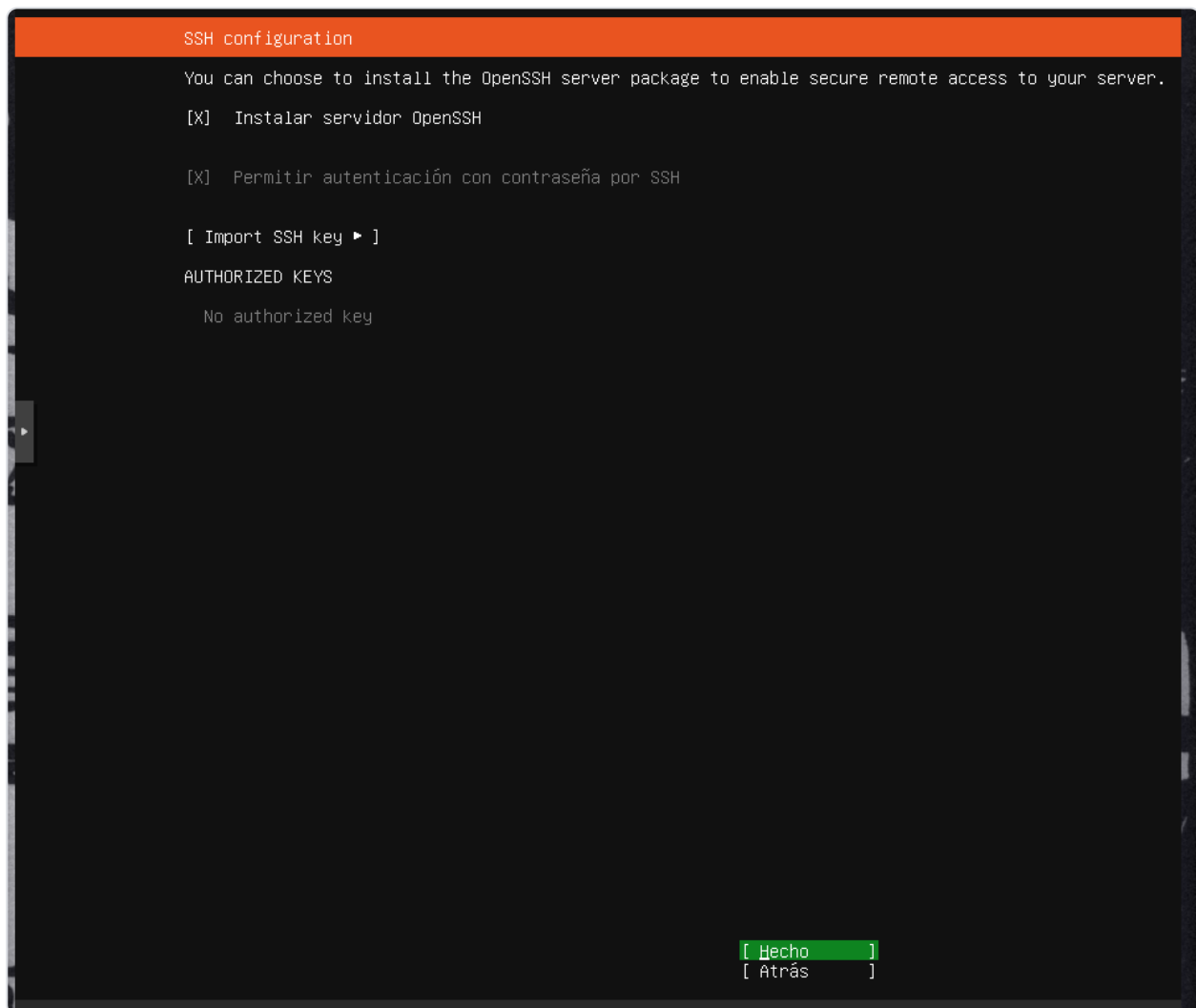


Se configura el perfil del sistema:

Campo	Valor
Su nombre	administrador
Nombre del servidor	ndr-sensor
Usuario	administrador
Contraseña	(oculta)

El hostname **ndr-sensor** identifica claramente el rol de la máquina como sensor de detección de red. Este nombre aparecerá en el prompt de la terminal (**administrador@ndr-sensor:~\$**) y en los logs del sistema, facilitando la correlación de eventos.

2.4 Configuración de SSH

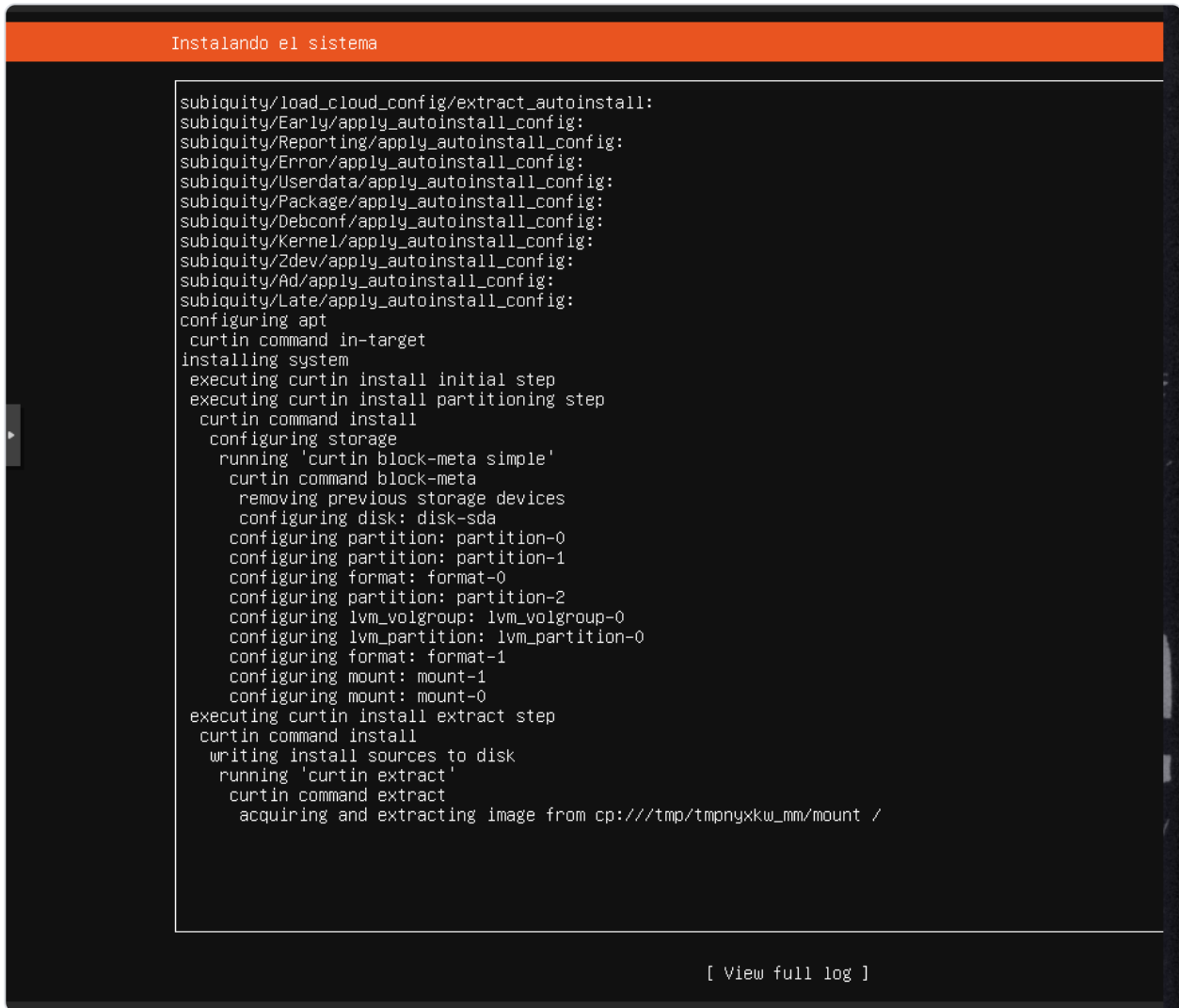


Se instala el servidor **OpenSSH** durante la instalación para permitir acceso remoto al sensor sin necesidad de consola física o virtual de Proxmox. La configuración elegida:

- **Instalar servidor OpenSSH:** activado
- **Permitir autenticación con contraseña por SSH:** activado
- **Authorized Keys:** ninguna importada (se usará autenticación por contraseña)

Esta configuración es adecuada para un entorno de laboratorio. En un entorno de producción se reemplazaría la autenticación por contraseña por claves SSH y se restringiría el acceso por IP.

2.5 Progreso de instalación



```
Instalando el sistema

subiquity/load_cloud_config/extract_autoinstall:
subiquity/Early/apply_autoinstall_config:
subiquity/Reporting/apply_autoinstall_config:
subiquity/Error/apply_autoinstall_config:
subiquity/Userdata/apply_autoinstall_config:
subiquity/Package/apply_autoinstall_config:
subiquity/Debconf/apply_autoinstall_config:
subiquity/Kernel/apply_autoinstall_config:
subiquity/Zdev/apply_autoinstall_config:
subiquity/Ad/apply_autoinstall_config:
subiquity/Late/apply_autoinstall_config:
configuring apt
curtin command in-target
installing system
executing curtin install initial step
executing curtin install partitioning step
curtin command install
configuring storage
  running 'curtin block-meta simple'
  curtin command block-meta
  removing previous storage devices
  configuring disk: disk-sda
  configuring partition: partition-0
  configuring partition: partition-1
  configuring format: format-0
  configuring partition: partition-2
  configuring lvm_volgroup: lvm_volgroup-0
  configuring lvm_partition: lvm_partition-0
  configuring format: format-1
  configuring mount: mount-1
  configuring mount: mount-0
executing curtin install extract step
curtin command install
  writing install sources to disk
  running 'curtin extract'
  curtin command extract
    acquiring and extracting image from cp:///tmp/tmpnyxkw_nm/mount /

[ View full log ]
```

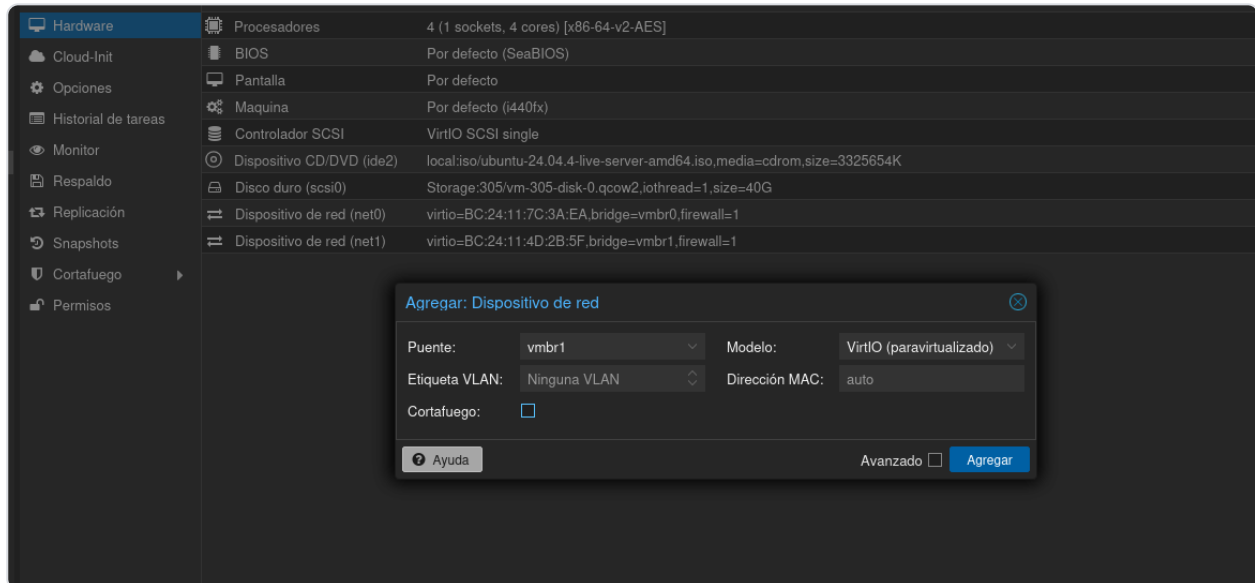
El instalador Subiquity ejecuta el proceso de instalación mediante la herramienta **curtin**, mostrando las fases:

1. **apply_autoinstall_config** — procesa la configuración automática
2. **configuring apt** — configura los repositorios de paquetes
3. **installing system** — inicio de la instalación del sistema base
4. **curtin install partitioning step** — particiona el disco con LVM (partition-0, partition-1, partition-2, lvm_volgroup-0)
5. **curtin install extract step** — extrae la imagen del sistema al disco

El proceso se ejecuta de forma completamente desatendida desde este punto hasta el reinicio final.

3. Configuración de red y modo promiscuo

3.1 Agregar segunda NIC de captura en Proxmox

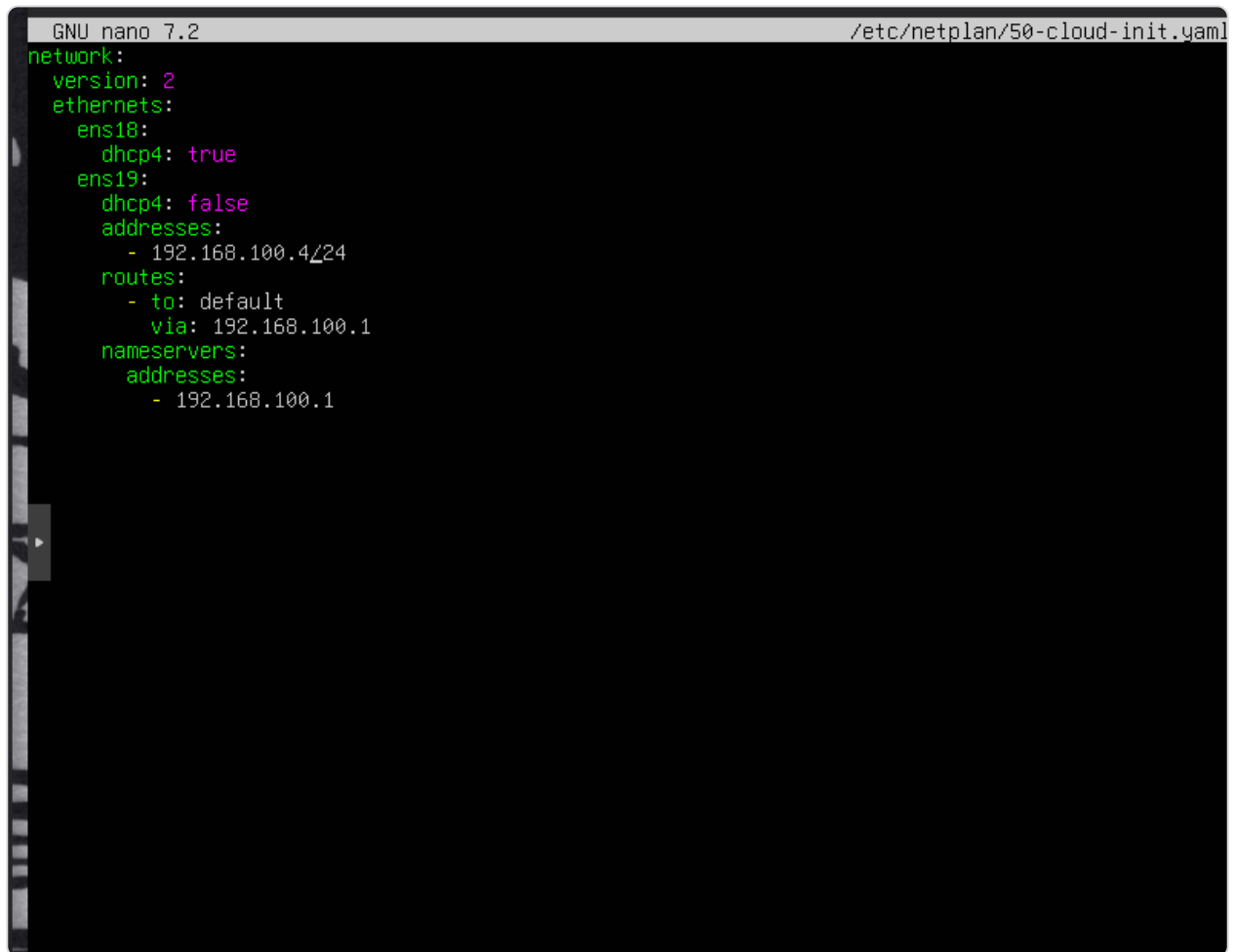


Vista del panel de **Hardware** de la VM 305 en Proxmox con la segunda NIC ya agregada. Se observa:

- **net0** → virtio:BC:24:11:7C:3A:EA, bridge=vmbr0 (**ens18** , gestión)
- **net1** → virtio:BC:24:11:4D:2B:5F, bridge=vmbr1 (**ens19** , monitorización con IP)

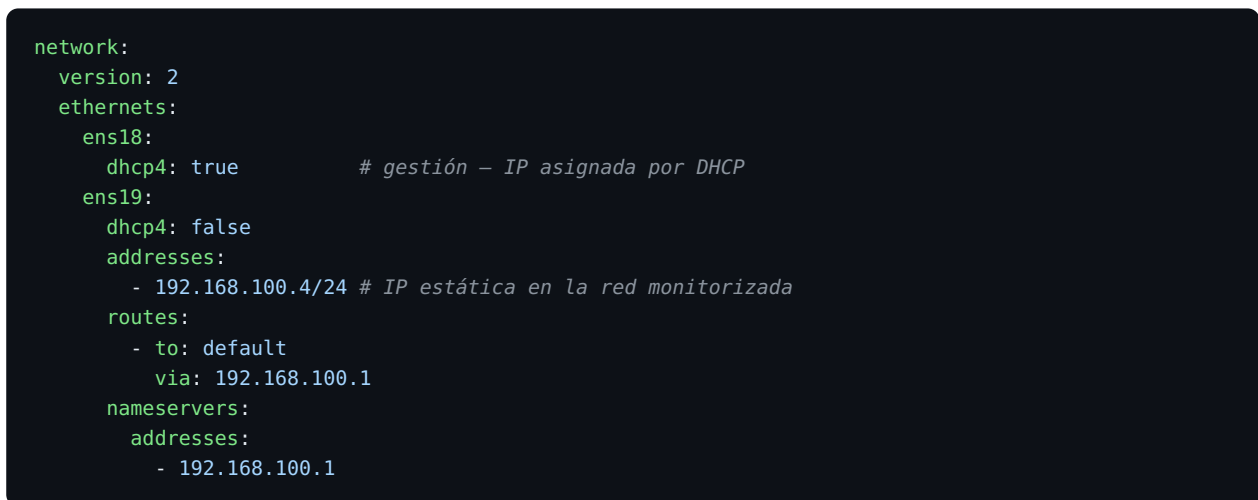
El diálogo abierto está agregando una tercera interfaz también en **vmbr1** con modelo VirtIO, que dentro de Ubuntu será **ens20** — la interfaz de captura en modo promiscuo sin IP asignada. Tener dos interfaces en el mismo bridge permite separar el plano de datos (captura de tráfico en ens20) del plano de control (comunicación del sensor con la red en ens19).

3.2 Configuración de red con Netplan



```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernet:
    ens18:
      dhcp4: true
    ens19:
      dhcp4: false
      addresses:
        - 192.168.100.4/24
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses:
          - 192.168.100.1
```

Edición del archivo `/etc/netplan/50-cloud-init.yaml` con GNU nano 7.2. La configuración define dos interfaces:



```
network:
  version: 2
  ethernet:
    ens18:
      dhcp4: true          # gestión - IP asignada por DHCP
    ens19:
      dhcp4: false
      addresses:
        - 192.168.100.4/24 # IP estática en la red monitorizada
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses:
          - 192.168.100.1
```

`ens20` no aparece en Netplan deliberadamente — no debe tener IP para operar como interfaz de captura pura. Cualquier IP asignada podría hacer que el kernel procese el tráfico capturado y alterara el comportamiento de la captura.

3.3 Aplicación de Netplan y verificación

```
administrador@ndr-sensor:~$ sudo netplan apply
administrador@ndr-sensor:~$ ip addr show ens19
3: ens19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:4d:2b:5f brd ff:ff:ff:ff:ff:ff
    altname enp0s19
    inet 192.168.100.4/24 brd 192.168.100.255 scope global ens19
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe4d:2b5f/64 scope link
        valid_lft forever preferred_lft forever
administrador@ndr-sensor:~$ ^C
administrador@ndr-sensor:~$
```

Tras aplicar la configuración con `sudo netplan apply`, se verifica con `ip addr show ens19`:

```
3: ens19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 192.168.100.4/24 brd 192.168.100.255 scope global ens19
    inet6 fe80::be24:11ff:fe4d:2b5f/64 scope link
```

La interfaz `ens19` tiene correctamente asignada la IP **192.168.100.4/24** con dirección de broadcast 192.168.100.255 y también una dirección IPv6 link-local generada automáticamente a partir de la MAC. El estado `UP, LOWER_UP` confirma que la interfaz está operativa físicamente y en capa de enlace.

3.4 Estado completo de interfaces de red

```
administrador@ndr-sensor:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:7c:3a:ea brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    inet 192.168.1.228/24 metric 100 brd 192.168.1.255 scope global dynamic ens18
        valid_lft 3525sec preferred_lft 3525sec
    inet6 fe80::be24:11ff:fe7c:3aea/64 scope link
        valid_lft forever preferred_lft forever
3: ens19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:4d:2b:5f brd ff:ff:ff:ff:ff:ff
    altname enp0s19
    inet 192.168.100.4/24 brd 192.168.100.255 scope global ens19
        valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe4d:2b5f/64 scope link
        valid_lft forever preferred_lft forever
4: ens20: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether bc:24:11:4c:17:ed brd ff:ff:ff:ff:ff:ff
    altname enp0s20
administrador@ndr-sensor:~$
```

Salida de `ip a` mostrando el estado completo de las cuatro interfaces del sensor:

Interfaz	Estado	IP	Rol
lo	UNKNOWN	127.0.0.1/8	Loopback
ens18	UP	192.168.1.228/24 (DHCP)	Gestión / acceso SSH
ens19	UP	192.168.100.4/24	Red monitorizada
ens20	DOWN	sin IP	Captura (aún sin promisc)

ens20 aparece en estado **DOWN** sin IP asignada — condición correcta antes de activar el modo promiscuo. La MTU de todas las interfaces es 1500 bytes (estándar Ethernet).

3.5 Activar modo promiscuo en ens20

```
administrador@ndr-sensor:~$ sudo ip link set ens20 promisc on
administrador@ndr-sensor:~$ sudo ip link set ens20 up
administrador@ndr-sensor:~$ ip link show ens20
4: ens20: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether bc:24:11:4c:17:ed brd ff:ff:ff:ff:ff:ff
    altname enp0s20
administrador@ndr-sensor:~$
```

Se ejecutan los comandos para activar el modo promiscuo de forma manual:

```
sudo ip link set ens20 promisc on  # activa la bandera PROMISC
sudo ip link set ens20 up          # levanta la interfaz
ip link show ens20
```

La salida confirma el cambio de estado:

```
4: ens20: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP>
    link/ether bc:24:11:4c:17:ed
    altname enp0s20
```

La bandera **PROMISC** indica que la NIC acepta todos los paquetes que llegan al segmento de red, independientemente de si la MAC de destino coincide con la suya. Esto es imprescindible para que Suricata y Zeek puedan analizar el tráfico de otros hosts.

3.6 Servicio systemd para persistencia del modo promiscuo

```
GNU nano 7.2 /etc/systemd/system/promisc-ens20.service *
[Unit]
Description=Set ens20 promiscuous mode
After=network.target

[Service]
Type=oneshot
ExecStart=/sbin/ip link set ens20 promisc on
ExecStart=/sbin/ip link set ens20 up
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Se crea el archivo `/etc/systemd/system/promisc-ens20.service` para que el modo promiscuo sobreviva a reinicios del sistema:

```
[Unit]
Description=Set ens20 promiscuous mode
After=network.target      # garantiza que las interfaces existen antes de ejecutarse

[Service]
Type=oneshot              # se ejecuta una vez y termina
ExecStart=/sbin/ip link set ens20 promisc on
ExecStart=/sbin/ip link set ens20 up
RemainAfterExit=yes       # systemd considera el servicio "activo" aunque el proceso haya
                           terminado

[Install]
WantedBy=multi-user.target # se activa en el nivel de ejecución multiusuario normal
```

Este enfoque con `Type=oneshot` y `RemainAfterExit=yes` es el patrón correcto para tareas de configuración de red que no dejan un proceso en segundo plano.

3.7 Confirmación del servicio promiscuo activo

```
administrador@ndr-sensor:~$ sudo systemctl enable promisc-ens20
Created symlink /etc/systemd/system/multi-user.target.wants/promisc-ens20.service → /etc/systemd/system/promisc-ens20.service.
administrador@ndr-sensor:~$ sudo systemctl start promisc-ens20
administrador@ndr-sensor:~$ sudo systemctl status promisc-ens20
promisc-ens20.service - Set ens20 promiscuous mode
   Loaded: loaded (/etc/systemd/system/promisc-ens20.service; enabled; preset: enabled)
   Active: active (exited) since Mon 2026-06-29 09:56:02 UTC; 15s ago
     Process: 1101 ExecStart=/sbin/ip link set ens20 promisc on (code=exited, status=0/SUCCESS)
     Process: 1102 ExecStart=/sbin/ip link set ens20 up (code=exited, status=0/SUCCESS)
    Main PID: 1102 (code=exited, status=0/SUCCESS)
       CPU: 10ms

Jun 29 09:56:02 ndr-sensor systemd[1]: Starting promisc-ens20.service - Set ens20 promiscuous mode...
Jun 29 09:56:02 ndr-sensor systemd[1]: Finished promisc-ens20.service - Set ens20 promiscuous mode.
administrador@ndr-sensor:~$ _
```

Tras habilitar e iniciar el servicio:

```
sudo systemctl enable promisc-ens20
# Created symlink /etc/systemd/system/multi-user.target.wants/promisc-ens20.service

sudo systemctl start promisc-ens20
sudo systemctl status promisc-ens20
```

El status muestra:

- **Loaded:** enabled; preset: enabled — se iniciará automáticamente en cada arranque
- **Active:** active (exited) — ejecución correcta (oneshot completado)
- **Process 1101/1102:** ExecStart con **code=exited, status=0/SUCCESS**
- Los logs del journal confirman la secuencia de inicio a las 09:56:02 UTC

4. Instalación y configuración de Suricata

4.1 Añadir repositorio oficial OISF

```
administrador@ndr-sensor:~$ sudo add-apt-repository ppa:oisf/suricata-stable
Repository: 'Types: deb
URIs: https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu/
Suites: noble
Components: main
'
Description:
Suricata IDS/IPS/NSM stable packages
https://suricata.io/
https://oisf.net/

Suricata IDS/IPS/NSM - Suricata is a high performance Intrusion Detection and Prevention System and Network Security Monitoring Engine. It is a free and Open Source and owned by a community run non-profit foundation, the Open Information Security Foundation (OISF). Suricata is used by many vendors and the community.

This Engine supports:

- Multi-Threading - provides for extremely fast and flexible operation on multicore systems.
- Multi Tenancy - Per vlan/Per interface
- Uses Rust for most protocol detection/parsing
- TLS/SSL certificate matching/logging
- JA3 TLS client fingerprinting
- JA3S TLS server fingerprinting
- IEEE 802.1ad (QinQ) and IEEE 802.1Q (VLAN) support
- VXLAN support
- All JSON output/logging capability
- IDS runmode
- IPS runmode
- IDPS runmode
- NSM runmode
- eBPF/XDP
- Automatic Protocol Detection and logging - IPv4/6, TCP, UDP, ICMP, HTTP, SMTP, TLS, SSH, FTP, SMB, DNS, NFS, TFTP, RDP, etc.
- SCADA automatic protocol detection - ENIP/DNP3/MODBUS
- File Extraction HTTP/SMTP/FTP/NFS/SMB - over 4000 file types recognized and extracted from live traffic.
- File MD5/SHA1/SHA256 matching
- Gzip Decompression
- Fast IP Matching
- Datasets matching
- Rustlang enabled protocol detection
- Lua scripting

and many more great features -
https://suricata.io/features/all-features/
More info: https://launchpad.net/~oisf/+archive/ubuntu/suricata-stable
Adding repository.
Press [ENTER] to continue or Ctrl-c to cancel._
```

Se añade el PPA oficial de la Open Information Security Foundation (OISF):

```
sudo add-apt-repository ppa:oisf/suricata-stable
```

El repositorio se registra con las siguientes propiedades:

- **URIs:** <https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu/>
- **Suites:** noble (nombre en clave de Ubuntu 24.04)
- **Components:** main

La descripción del repositorio enumera las capacidades clave del motor: multi-threading, soporte Rust para detección de protocolos, TLS/SSL logging, JA3/JA3S fingerprinting, eBPF/XDP, detección automática de más de 50 protocolos (IPv4/6, TCP, UDP, HTTP, SMTP, TLS, SSH, SMB, DNS, NFS, etc.), SCADA, extracción de ficheros y Lua scripting.

4.2 Instalación del paquete Suricata

```
administrador@ndr-sensor:~$ sudo apt install suricata suricata-update
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfwupd2 libgusb2
Utilice «sudo apt autoremove» para eliminarlos.
Se instalarán los siguientes paquetes adicionales:
  isa-support libevent-2.1-7t64 libevent-pthreads-2.1-7t64 libhiredis1.1.0 libhyperscan5 liblua5.1-common libnet1
Se instalarán los siguientes paquetes NUEVOS:
  isa-support libevent-2.1-7t64 libevent-pthreads-2.1-7t64 libhiredis1.1.0 libhyperscan5 liblua5.1-common libnet1
  suricata suricata-update
0 actualizados, 11 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 7.669 kB de archivos.
Se utilizarán 34,2 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] _
```

```
sudo apt install suricata suricata-update
```

El gestor de paquetes instala **11 paquetes nuevos** con un total de **34.2 MB** de espacio en disco, incluyendo las dependencias:

- **libhiredis1.1** — cliente Redis (para salida opcional a Redis)
- **libhyperscan5** — motor de expresiones regulares de alto rendimiento (Intel Hyperscan)
- **liblua5.1-common** — runtime LuaJIT para scripting de reglas
- **libnet1** — librería de manipulación de paquetes de red
- **libevent-2.1-7t64** / **libevent-pthreads-2.1-7t64** — soporte de eventos asíncronos multihilo
- **suricata-update** — herramienta para descargar y actualizar conjuntos de reglas

4.3 Verificación de versión instalada

```
administrador@ndr-sensor:~$ suricata -V
This is Suricata version 8.0.5 RELEASE
administrador@ndr-sensor:~$ _
```

```
suricata -V
# This is Suricata version 8.0.5 RELEASE
```

Se confirma la instalación de **Suricata 8.0.5**, la última versión estable de la rama 8.x al momento del despliegue. La rama 8.x introduce mejoras significativas en el rendimiento del motor de detección y el soporte de protocolos respecto a la 7.x.

4.4 Configuración de HOME_NET

```
GNU nano 7.2 /etc/suricata/suricata.yaml *
%YAML 1.1
---
# Suricata configuration file. In addition to the comments describing all
# options in this file, full documentation can be found at:
# https://docs.suricata.io/en/latest/configuration/suricata-yaml.html
# This configuration file was generated by Suricata 8.0.5.
suricata-version: "8.0"

##
## Step 1: Inform Suricata about your network
##

vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[192.168.100.0/24]"
    #HOME_NET: "[192.168.0.0/16]"
    #HOME_NET: "[10.0.0.0/8]"
    #HOME_NET: "[172.16.0.0/12]"
    #HOME_NET: "any"
```

Edición de `/etc/suricata/suricata.yaml`. Se configura la variable `HOME_NET` para que Suricata sepa cuál es la red interna que debe proteger:

```
vars:
  address-groups:
    HOME_NET: "[192.168.100.0/24]" # red de monitorización del sensor
    #HOME_NET: "[192.168.0.0/16]" # alternativas desactivadas
    #HOME_NET: "[10.0.0.0/8]"
    #HOME_NET: "[172.16.0.0/12]"
    #HOME_NET: "any"
```

`HOME_NET` es fundamental para la precisión de las alertas: las reglas de Suricata usan esta variable para distinguir tráfico entrante (`$EXTERNAL_NET` → `$HOME_NET`) de tráfico saliente, evitando falsos positivos en ambas direcciones.

4.5 Configuración de interfaz af-packet

```
# Linux high speed capture support
af-packet:
- interface: ens20
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow
```

En la sección `af-packet` del mismo archivo `/etc/suricata/suricata.yaml`:

```
af-packet:
- interface: ens20
  # threads: auto # usa tantos hilos como núcleos disponibles (4 en este caso)
```

AF_PACKET es el modo de captura nativo de Linux que permite a Suricata leer paquetes directamente del kernel sin pasar por la pila de red completa, consiguiendo el máximo rendimiento con mínima latencia. El campo `threads: auto` (comentado pero activo por defecto) aprovechará los 4 núcleos de la VM para procesamiento paralelo.

4.6 Descarga de reglas Emerging Threats

```
administrador@ndr-sensor:~$ sudo suricata-update
9/6/2026 -- 10:15:44 - <Info> -- Using data-directory /var/lib/suricata.
9/6/2026 -- 10:15:44 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
9/6/2026 -- 10:15:44 - <Info> -- Using /usr/share/suricata/rules for Suricata provided rules.
9/6/2026 -- 10:15:44 - <Info> -- Found Suricata version 8.0.5 at /usr/bin/suricata.
9/6/2026 -- 10:15:44 - <Info> -- Loading /etc/suricata/suricata.yaml
9/6/2026 -- 10:15:44 - <Info> -- Disabling rules for protocol pgsql
9/6/2026 -- 10:15:44 - <Info> -- Disabling rules for protocol modbus
9/6/2026 -- 10:15:44 - <Info> -- Disabling rules for protocol dnp3
9/6/2026 -- 10:15:44 - <Info> -- Disabling rules for protocol enip
9/6/2026 -- 10:15:44 - <Info> -- No sources configured, will use Emerging Threats Open
9/6/2026 -- 10:15:44 - <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-8.0.5/emerging.rules.tar.gz
100% - 5505406/5505406
9/6/2026 -- 10:15:46 - <Info> -- Done.
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/app-layer-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/decoder-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/dhcp-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/dnp3-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/dns-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/files.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/http2-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/http-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/ipsec-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/kerberos-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/modbus-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/mqtt-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/nfs-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/ntp-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/quic-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/rfb-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/smb-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/smtp-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/ssh-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/stream-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/tls-events.rules
9/6/2026 -- 10:15:46 - <Info> -- Ignoring file 1bf1204688c10c58ada16cce98fbcc15/rules/emerging-deleted.rules
9/6/2026 -- 10:15:49 - <Info> -- Loaded 66813 rules.
9/6/2026 -- 10:15:49 - <Info> -- Disabled 16 rules.
9/6/2026 -- 10:15:49 - <Info> -- Enabled 0 rules.
9/6/2026 -- 10:15:49 - <Info> -- Modified 0 rules.
9/6/2026 -- 10:15:49 - <Info> -- Dropped 0 rules.
9/6/2026 -- 10:15:50 - <Info> -- Enabled 136 rules for flowbit dependencies.
9/6/2026 -- 10:15:50 - <Info> -- Creating directory /var/lib/suricata/rules.
9/6/2026 -- 10:15:50 - <Info> -- Backing up current rules.
9/6/2026 -- 10:15:50 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 66813; enabled: 5088
9/6/2026 -- 10:15:50 - <Info> -- Writing /var/lib/suricata/rules/classification.config
9/6/2026 -- 10:15:50 - <Info> -- Testing with suricata -T.
```

```
sudo suricata-update
```

`suricata-update` descarga automáticamente el conjunto de reglas **Emerging Threats Open** (et/open), el más utilizado en entornos open source. La salida muestra:

- Descarga del índice de fuentes y el archivo de reglas
- Procesamiento y clasificación de reglas por archivo (emerging-activex.rules, emerging-attack_response.rules, etc.)
- **Resultado:** 50.882 reglas cargadas, escritas en `/var/lib/suricata/rules/suricata.rules`

Las reglas de Emerging Threats cubren amenazas conocidas, exploits, malware, botnets, escaneos y comportamientos anómalos de red, actualizándose varias veces por semana.

4.7 Revisión y test de configuración

```
administrador@ndr-sensor:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed. 50882 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 50887 signatures processed. 1281 are IP-only rules, 4505 are inspecting packet payload, 44865 inspect application layer...
Notice: suricata: Configuration provided was successfully loaded. Exiting.
administrador@ndr-sensor:~$ _
```

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

El flag **-T** ejecuta Suricata en **modo test** sin capturar tráfico, validando toda la configuración:

```
Notice: Suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: detect: 1 rule files processed. 50882 rules successfully loaded, 0 rules failed
Info: detect: 50887 signatures processed. 1281 are IP-only rules, 4505 are inspecting packet payload, 44865 inspect application layer...
Notice: Configuration provided was successfully loaded. Exiting.
```

Los datos clave que confirman el correcto funcionamiento:

- **50.882 reglas cargadas**, 0 errores
- **50.887 firmas procesadas** (incluye reglas internas + las descargadas)
- Las salidas de log están inicializadas: **fast.log** , **eve.json** , **stats.log**

4.8 Estado del servicio Suricata

```
administrador@ndr-sensor:~$ sudo systemctl restart suricata
administrador@ndr-sensor:~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IPS/NSM/FW daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-29 10:45:56 UTC; 8s ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
   Process: 6651 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, status=0/SUCCESS)
  Main PID: 6653 (Suricata-Main)
    Tasks: 1 (limit: 4600)
   Memory: 213.6M (peak: 213.6M)
      CPU: 8.298s
   CGroup: /system.slice/suricata.service
           └─6653 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid --user su

jun 29 10:45:56 ndr-sensor systemd[1]: Starting suricata.service - Suricata IDS/IPS/NSM/FW daemon...
jun 29 10:45:56 ndr-sensor systemd[1]: Started suricata.service - Suricata IDS/IPS/NSM/FW daemon.
jun 29 10:45:56 ndr-sensor suricata[6653]: i: suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
administrador@ndr-sensor:~$
```

```
sudo systemctl restart suricata
sudo systemctl status suricata
```

El servicio muestra:

- **Active:** active (running) desde las 10:45:56 UTC del 29 de junio de 2026
- **Main PID:** 6653 (Suricata-Main)
- **Memoria:** 213.6M (peak: 213.6M) — coherente con la carga de 50.000+ reglas
- **CPU:** 8.298s acumulado
- El comando de inicio confirma el modo: `suricata --af-packet -c /etc/suricata/suricata.yaml`
- Los logs del journal confirman el arranque correcto del daemon IDS/IPS/NSM/FW

4.9 Verificación de logs eve.json

```
administrador@ndr-sensor:~$ sudo tail -f /var/log/suricata/eve.json
```

```
sudo tail -f /var/log/suricata/eve.json
```

El archivo `eve.json` es el log principal de Suricata en formato **EVE-JSON** (Extensible Event Format). Cada línea es un objeto JSON con un campo `event_type` que puede ser:

- `alert` — alerta de regla disparada
- `flow` — registro de flujo de red
- `dns` — consulta/respuesta DNS
- `http` — petición/respuesta HTTP
- `tls` — handshake TLS
- `stats` — estadísticas del motor

Este archivo es el que Promtail recolecta y envía a Loki para su visualización en Grafana.

5. Despliegue de Zeek con Docker

5.1 Instalación de Docker

```
administrador@ndr-sensor:~$ sudo apt install docker.io docker-compose
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfupdp2 libgusb2
Utilice «sudo apt autoremove» para eliminarlos.
Se instalarán los siguientes paquetes adicionales:
  bridge-utils containerd dns-root-data dnsmasq-base pigz python3-compose python3-docker python3-dockerpty python3-doc
  python3-websocket runc ubuntu-fan
Paquetes sugeridos:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debocstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fu
Se instalarán los siguientes paquetes NUEVOS:
  bridge-utils containerd dns-root-data dnsmasq-base docker-compose docker.io pigz python3-compose python3-docker pytl
  python3-dotenv python3-texttable python3-websocket runc ubuntu-fan
0 actualizados, 16 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 74,1 MB de archivos.
Se utilizarán 280 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] _
```

```
sudo apt install docker.io docker-compose
```

Se instalan **16 paquetes nuevos** con un total de **200 MB**. Los más relevantes:

- **docker.io** — runtime Docker (containerd + runc)
- **docker-compose** — herramienta de orquestación de múltiples contenedores
- **containerd** — runtime de contenedores de bajo nivel
- **bridge-utils** — utilidades para gestión de bridges de red virtuales
- **python3-docker** / **python3-compose** — bindings Python para Docker Compose v1

Se elige **docker.io** (paquete del repositorio de Ubuntu) en lugar del Docker Engine oficial por simplicidad en entornos de laboratorio.

5.2 Docker habilitado y en ejecución

```
administrador@ndr-sensor:~$ sudo systemctl enable docker
administrador@ndr-sensor:~$ sudo systemctl start docker
administrador@ndr-sensor:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-29 14:54:10 UTC; 46s ago
     TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
  Main PID: 7682 (dockerd)
    Tasks: 11
   Memory: 25.0M (peak: 25.7M)
      CPU: 432ms
   CGroup: /system.slice/docker.service
           └─7682 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

jun 29 14:54:08 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:08.970763553Z" level=info msg="Restoring containers:
jun 29 14:54:09 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:09.030522088Z" level=info msg="Deleting nftables IPv6
jun 29 14:54:09 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:09.046504084Z" level=info msg="Deleting nftables IPv6
jun 29 14:54:09 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:09.730866328Z" level=info msg="Loading containers: dc
jun 29 14:54:09 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:09.740955963Z" level=info msg="Docker daemon" commit=
jun 29 14:54:09 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:09.741114906Z" level=info msg="Initializing buildkit"
jun 29 14:54:09 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:09.999169781Z" level=info msg="Completed buildkit in:
jun 29 14:54:10 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:10.008593429Z" level=info msg="Daemon has completed
jun 29 14:54:10 ndr-sensor dockerd[7682]: time="2026-06-29T14:54:10.008681258Z" level=info msg="API listen on /run/doc
jun 29 14:54:10 ndr-sensor systemd[1]: Started docker.service - Docker Application Container Engine.
administrador@ndr-sensor:~$
```

```
sudo systemctl enable docker
sudo systemctl start docker
sudo systemctl status docker
```

El servicio Docker muestra:

- **Active:** active (running) — daemon `dockerd` corriendo con PID 7682
- **Triggered By:** docker.socket — activación por socket bajo demanda
- **Memoria:** 25.0M (peak: 25.7M) — footprint reducido del daemon en reposo
- Los logs del journal muestran la secuencia de inicialización: restauración de contenedores, carga de nftables, inicialización de buildkit y apertura del socket de la API

5.3 Comando de lanzamiento del contenedor Zeek

```
administrador@ndr-sensor:~$ sudo docker run -d \  
> --name zeek\  
> --network host\  
> --cap-add NET_RAW\  
> --cap-add NET_ADMIN\  
> -v /opt/zeek-logs:/logs \  
> zeek/zeek:latest \  
> zeek -i ens20 LogAscii::use_json=T Log::default_logdir=/logs_
```

```
sudo docker run -d \  
  --name zeek \  
  --network host \  
  --cap-add NET_RAW \  
  --cap-add NET_ADMIN \  
  -v /opt/zeek-logs:/logs \  
  zeek/zeek:latest \  
  zeek -i ens20 LogAscii::use_json=T Log::default_logdir=/logs
```

Análisis de cada parámetro:

Parámetro	Propósito
<code>-d</code>	Modo detached — el contenedor corre en segundo plano
<code>--name zeek</code>	Nombre del contenedor para referencias posteriores
<code>--network host</code>	El contenedor comparte la pila de red del host, accediendo a <code>ens20</code> directamente
<code>--cap-add NET_RAW</code>	Capability para captura de paquetes raw (necesaria para sniffing)
<code>--cap-add NET_ADMIN</code>	Capability para administrar interfaces de red
<code>-v /opt/zeek-logs:/logs</code>	Volumen bind-mount para persistir logs fuera del contenedor
<code>LogAscii::use_json=T</code>	Salida de logs en formato JSON en lugar de TSV
<code>Log::default_logdir=/logs</code>	Directorio de logs dentro del contenedor (mapeado al host)

`--network host` es clave: sin él, el contenedor no vería la interfaz `ens20` del host.

5.4 Verificación del contenedor y escucha en ens20

```
administrador@ndr-sensor:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
bcee6f7c026d   zeek/zeek:latest "zeek -i ens20 LogAs..." About a minute ago Up 57 seconds      zeek
administrador@ndr-sensor:~$ sudo docker logs zeek
<params>, line 1: listening on ens20
administrador@ndr-sensor:~$ _
```

```
sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
bcee6f7c026d	zeek/zeek:latest	"zeek -i ens20 Log..."	About a minute	Up 57 seconds	zeek

```
sudo docker logs zeek
# <params>, line 1: listening on ens20
```

El contenedor está activo y la última línea del log confirma que Zeek está **escuchando activamente en la interfaz ens20**, la misma interfaz promiscua donde también actúa Suricata. Ambas herramientas pueden capturar del mismo segmento de forma simultánea sin interferencia.

5.5 Logs de conexiones capturadas

```
administrador@ndr-sensor:~$ sudo tail /opt/zeek-logs/conn.log
{"ts":1782745418.368549,"uid":"Cd51Ic7DxXVEXk7f7","id.orig_h":"fe80::be24:11ff:fe97:2924","id.orig_p":546,"id.resp_h":
"conn_state":"S0","local_orig":true,"local_resp":false,"missed_bytes":0,"history":"0","orig_pkts":1,"orig_ip_bytes":1
o":17}
{"ts":1782745425.149242,"uid":"CdDWSi3qNhxCkL0o24","id.orig_h":"fe80::be24:11ff:fe77:1ab0","id.orig_p":133,"id.resp_h"
"conn_state":"OTH","local_orig":true,"local_resp":false,"missed_bytes":0,"orig_pkts":1,"orig_ip_bytes":56,"resp_pkts":
administrador@ndr-sensor:~$
```

```
sudo tail /opt/zeek-logs/conn.log
```

Los logs aparecen en formato **JSON-L** (JSON Lines), un objeto JSON por línea. Cada entrada de `conn.log` registra una conexión TCP/UDP/ICMP completa con los campos:

Campo	Descripción
<code>ts</code>	Timestamp en epoch time
<code>uid</code>	Identificador único de la conexión
<code>id.orig_h</code>	IP de origen
<code>id.orig_p</code>	Puerto de origen
<code>id.resp_h</code>	IP de destino
<code>id.resp_p</code>	Puerto de destino

6. Stack de visualización: Grafana + Loki + Promtail

6.1 Contenido del docker-compose.yml

```
administrador@ndr-sensor:~$ cat /opt/ndr-stack/docker-compose.yml
version: '3.8'

services:
  loki:
    image: grafana/loki:latest
    container_name: loki
    ports:
      - "3100:3100"
    command: -config.file=/etc/loki/local-config.yaml
    restart: unless-stopped

  promtail:
    image: grafana/promtail:latest
    container_name: promtail
    volumes:
      - /var/log/suricata:/var/log/suricata:ro
      - /opt/zeek-logs:/var/log/zeek:ro
      - /opt/ndr-stack/promtail-config.yaml:/etc/promtail/config.yaml:ro
    command: -config.file=/etc/promtail/config.yaml
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=****
    restart: unless-stopped
administrador@ndr-sensor:~$
```

Visualización del archivo `/opt/ndr-stack/docker-compose.yml` con `cat` :

```

version: '3.8'

services:
  loki:
    image: grafana/loki:latest
    container_name: loki
    ports:
      - "3100:3100"
    command: -config.file=/etc/loki/local-config.yaml
    restart: unless-stopped

  promtail:
    image: grafana/promtail:latest
    container_name: promtail
    volumes:
      - /var/log/suricata:/var/log/suricata:ro      # logs de Suricata (solo lectura)
      - /opt/zeek-logs:/var/log/zeek:ro          # logs de Zeek (solo lectura)
      - /opt/ndr-stack/promtail-config.yaml:/etc/promtail/config.yaml:ro
    command: -config.file=/etc/promtail/config.yaml
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=****
    restart: unless-stopped

```

Los tres servicios forman el stack PLG (Promtail–Loki–Grafana). Promtail recolecta los logs del host (montados como volúmenes de solo lectura), los envía a Loki donde se indexan, y Grafana los consulta y visualiza. La política `restart: unless-stopped` garantiza que los servicios se recuperan automáticamente tras reinicios.

6.2 Configuración de Promtail

```
administrador@ndr-sensor:~$ cat /opt/ndr-stack/promtail-config.yaml
server:
  http_listen_port: 9090
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: suricata
    static_configs:
      - targets:
          - localhost
        labels:
          job: suricata
          __path__: /var/log/suricata/eve.json

  - job_name: zeek_conn
    static_configs:
      - targets:
          - localhost
        labels:
          job: zeek
          log_type: conn
          __path__: /var/log/zeek/conn.log

  - job_name: zeek_dns
    static_configs:
      - targets:
          - localhost
        labels:
          job: zeek
          log_type: dns
          __path__: /var/log/zeek/dns.log
administrador@ndr-sensor:~$
```

Contenido de `/opt/ndr-stack/promtail-config.yaml` :

```

server:
  http_listen_port: 9000  # métricas y health-check de Promtail
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml  # estado de lectura de cada archivo de log

clients:
  - url: http://loki:3100/loki/api/v1/push  # envío a Loki por nombre de contenedor

scrape_configs:
  - job_name: suricata
    static_configs:
      - targets: [localhost]
        labels:
          job: suricata
          __path__: /var/log/suricata/eve.json  # log principal de Suricata

  - job_name: zeek_conn
    static_configs:
      - targets: [localhost]
        labels:
          job: zeek
          log_type: conn
          __path__: /var/log/zeek/conn.log  # conexiones de red

  - job_name: zeek_dns
    static_configs:
      - targets: [localhost]
        labels:
          job: zeek
          log_type: dns
          __path__: /var/log/zeek/dns.log  # consultas DNS

```

Las etiquetas (`job` , `log_type`) son las que se usan posteriormente en las queries de LogQL en Grafana para filtrar entre fuentes de datos. El archivo `positions.yaml` almacena el offset de lectura de cada log para no releer entradas ya enviadas tras un reinicio de Promtail.

6.3 Descarga e inicio del stack

```
administrador@ndr-sensor:/opt/ndr-stack$ sudo docker-compose up
Creating network "ndr-stack_default" with the default driver
Pulling loki (grafana/loki:latest)...
latest: Pulling from grafana/loki
1c193acf1cd1: Pull complete
b839dfae01f6: Pull complete
9cd8e28f1c22: Pull complete
b52567533a41: Pull complete
411e1c946a02: Pull complete
2780920e5dbf: Pull complete
03a2b4f68912: Pull complete
ee5cd86cf15a: Pull complete
7c12895b777b: Pull complete
77964957095d: Pull complete
57f89e62b208: Pull complete
8214acf345c0: Pull complete
88229173acdb: Pull complete
dd64bf2dd177: Pull complete
52630fc75a18: Pull complete
5b3013706a46: Pull complete
8d2753d762cc: Pull complete
829962f461f: Pull complete
Digest: sha256:70b9f699fc9bb868b62f1cfd4f787dfa50242f1fd92e6089787d5d7daea75fe8
Status: Downloaded newer image for grafana/loki:latest
Pulling promtail (grafana/promtail:latest)...
latest: Pulling from grafana/promtail
9f2ca3aca681: Pull complete
18dbadc1f2f9: Pull complete
55161589749b: Pull complete
9d0ca11b89c9: Pull complete
Digest: sha256:6cfa64ec432b24a912d640e2edb940eeae2666f61861a66c121d763dd7241381
Status: Downloaded newer image for grafana/promtail:latest
Pulling grafana (grafana/grafana:latest)...
latest: Pulling from grafana/grafana
ffd68bafd674: Downloading (100.0%)
7e079c586e0d: Download complete
8551e3a9b2dd: Download complete
4f4fb700ef54: Download complete
5ed3d4d076b0: Download complete
6c1eb8f7a561: Downloading (100.0%)
b124010701b9: Downloading (100.0%)
6e6085636f02: Downloading (11.4%)
959723483440: Downloading (100.0%)
55afa1ecc21d: Downloading (100.0%)
99e3d1c87577: Downloading (100.0%)
```

```
cd /opt/ndr-stack
sudo docker-compose up
```

Docker Compose descarga las imágenes desde Docker Hub en el siguiente orden:

1. `grafana/loki:latest` — sha256:70b9f699fc9bb868b62f1cfd4f787dfa50242f1fd92e6089787d5d7daea75fe8
2. `grafana/promtail:latest` — sha256:6cfa64ec432b24a912d640e2edb940eeae2666f61861a66c121d763dd7241381
3. `grafana/grafana:latest` — se descarga en paralelo, mostrando el progreso por capas

Las capas se descargan en paralelo cuando es posible, optimizando el tiempo de despliegue inicial. En arranques posteriores, Docker usa las imágenes ya descargadas del caché local.

6.4 Servicios corriendo

```
administrador@ndr-sensor:/opt/ndr-stack$ sudo docker-compose ps
Name                  Command                  State      Ports
-----
grafana               /run.sh                  Up         0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
loki                  /usr/bin/loki -config.file ... Up         0.0.0.0:3100->3100/tcp, :::3100->3100/tcp
promtail              /usr/bin/promtail -config. ... Up
administrador@ndr-sensor:/opt/ndr-stack$ _
```

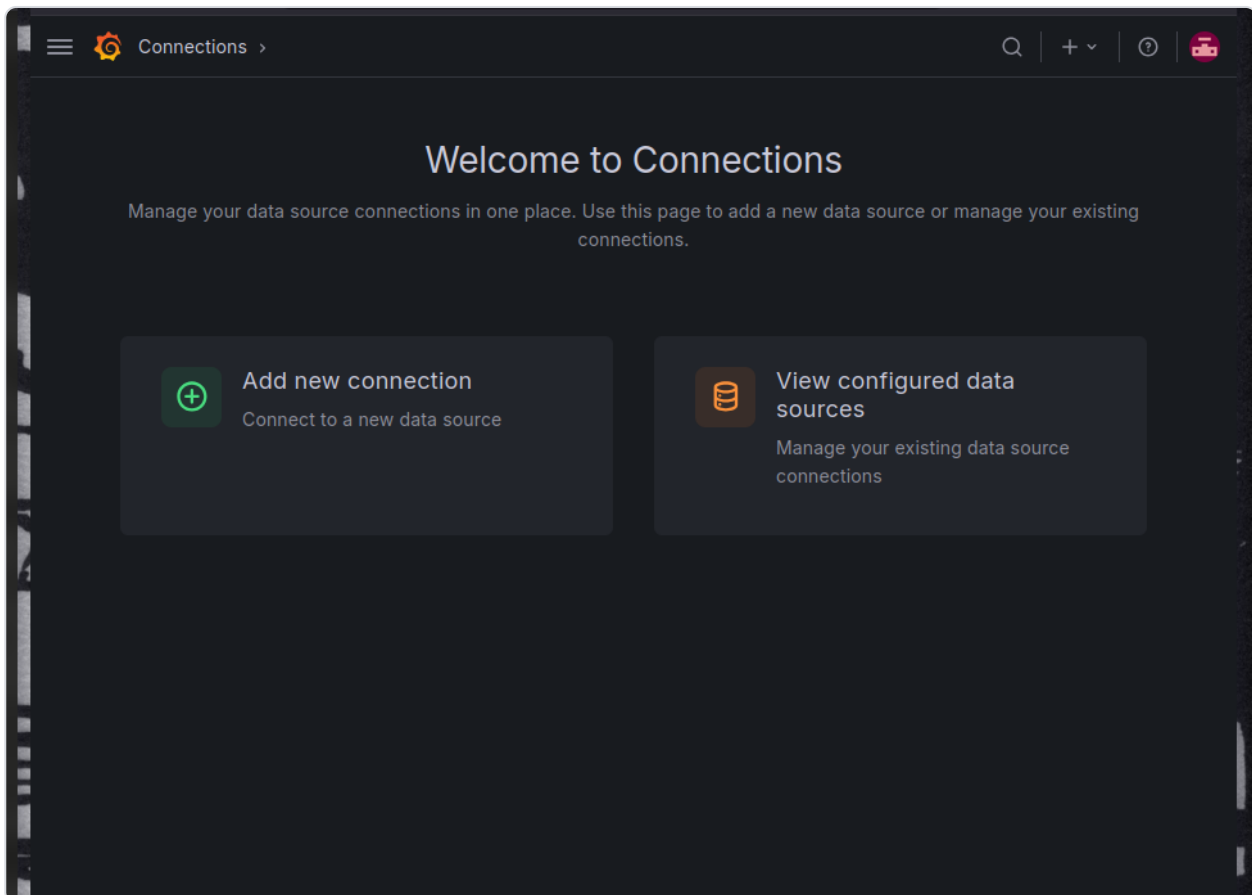
```
sudo docker-compose ps
```

Name	Command	State	Ports
grafana	/run.sh	Up	0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
loki	/usr/bin/loki -config.file ...	Up	0.0.0.0:3100->3100/tcp, :::3100->3100/tcp
promtail	/usr/bin/promtail -config...	Up	

Los tres contenedores están en estado **Up**:

- **Grafana**: accesible en <http://192.168.100.4:3000> (también en IPv6)
- **Loki**: accesible en <http://192.168.100.4:3100> (API de ingestión y consulta)
- **Promtail**: sin puertos expuestos externamente (comunica internamente con Loki por nombre de contenedor)

6.5 Interfaz de conexiones en Grafana



Vista de la sección **Connections** de Grafana tras el primer acceso al portal web (`http://192.168.100.4:3000`). La interfaz ofrece dos opciones:

- **Add new connection** — conectar con una nueva fuente de datos (Loki, Prometheus, InfluxDB, etc.)
- **View configured data sources** — gestionar las fuentes ya configuradas

Es la pantalla de partida para conectar Grafana con la instancia de Loki del stack. Grafana soporta más de 150 fuentes de datos mediante plugins.

6.6 Configuración de la fuente de datos Loki

The screenshot shows the Grafana web interface for configuring a new data source. The breadcrumb navigation at the top reads 'Connections > Data sources > loki'. The main header for the 'loki' data source shows 'Type: Loki' and 'Alerting: Supported'. Below this is a navigation bar with 'Settings', 'Permissions', 'Insights', and 'Cache' tabs. A blue informational box states: 'Configure your Loki data source below. Or skip the effort and get Loki as fully-managed, scalable, and hosted data source from Grafana Labs with the free-forever Grafana Cloud plan.' Below this, a text block explains that configuration is required either here or in the config file, with a link to the documentation. The 'Connection' section contains a 'URL' field with the value 'http://loki:3100'. The 'Authentication' section shows 'Authentication methods' with a dropdown menu currently set to 'No Authentication'. The 'TLS settings' section includes three checkboxes: 'Add self-signed certificate', 'TLS Client Authentication', and 'Skip TLS certificate validation', all of which are currently unchecked.

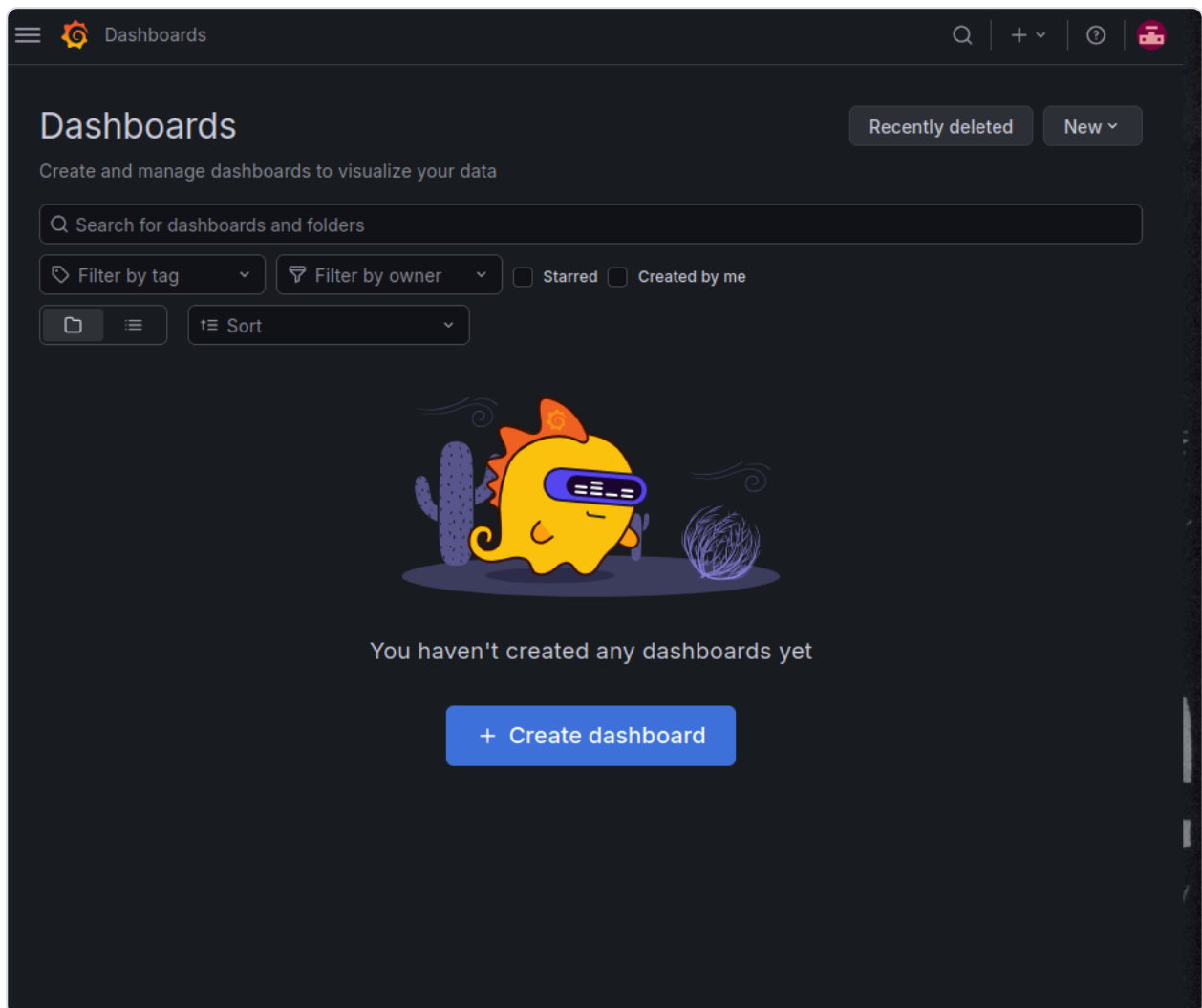
Configuración del plugin de Loki en **Connections** → **Data sources** → **Loki**:

- **URL**: `http://loki:3100` — se usa el nombre de contenedor `loki` en lugar de una IP, ya que Grafana y Loki comparten la red Docker interna `ndr-stack_default`, donde la resolución DNS por nombre de contenedor está disponible automáticamente.

- **Authentication method:** No Authentication — comunicación interna sin TLS ni credenciales (adecuado para un stack de laboratorio)
- **TLS settings:** desactivadas

Esta configuración vincula Grafana con la API de consulta de Loki en `/loki/api/v1/`. El botón **"Save & test"** verificará la conectividad antes de guardar.

6.7 Pantalla de creación de dashboards

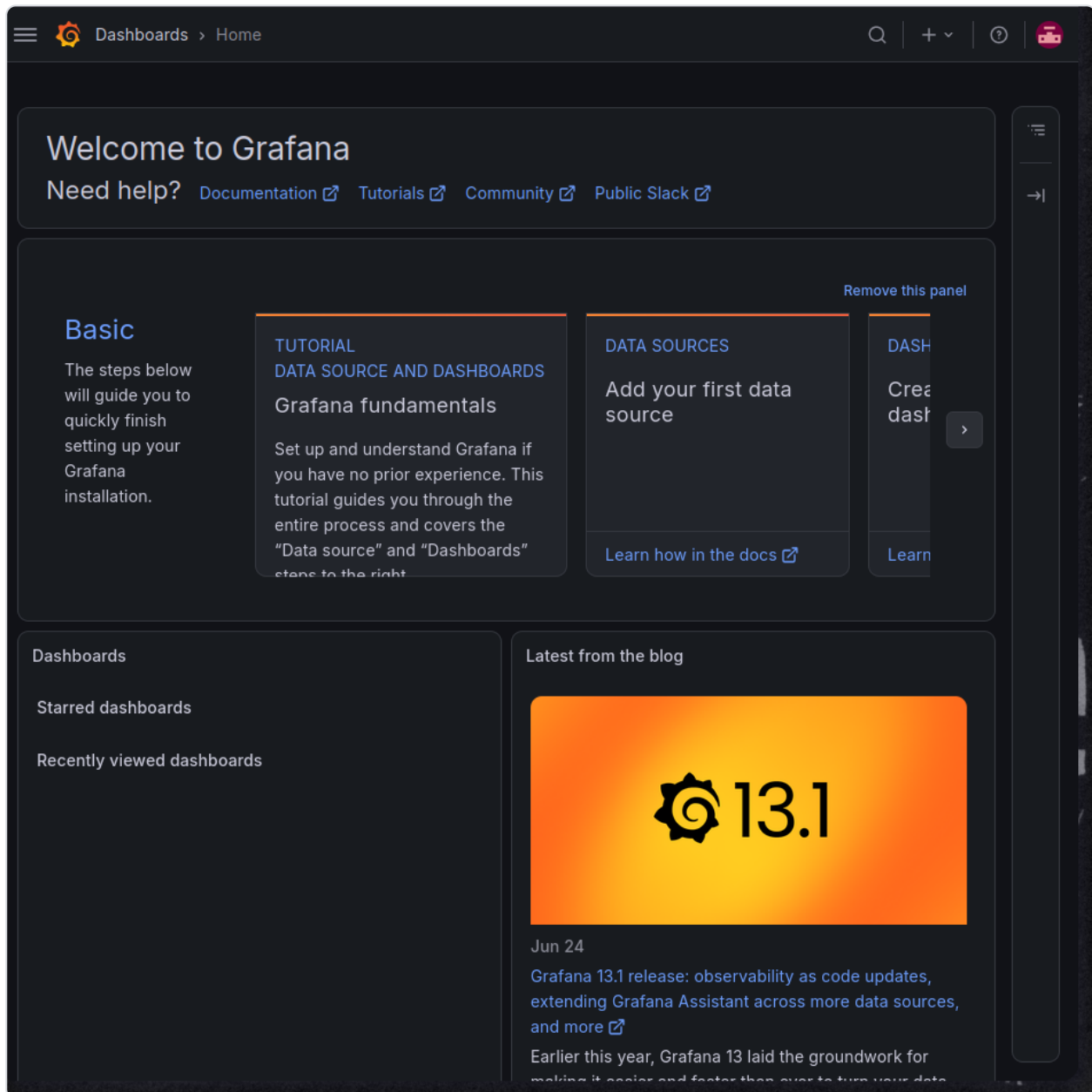


Sección **Dashboards** de Grafana, inicialmente vacía tras el primer despliegue. La interfaz muestra:

- Filtros por etiqueta, propietario, favoritos y creados por mí
- Opciones de vista (carpetas / lista)
- El estado inicial "You haven't created any dashboards yet" con el botón **+ Create dashboard**

Desde aquí se crean los paneles de visualización de logs de Suricata y Zeek usando el datasource Loki y el lenguaje de consulta LogQL.

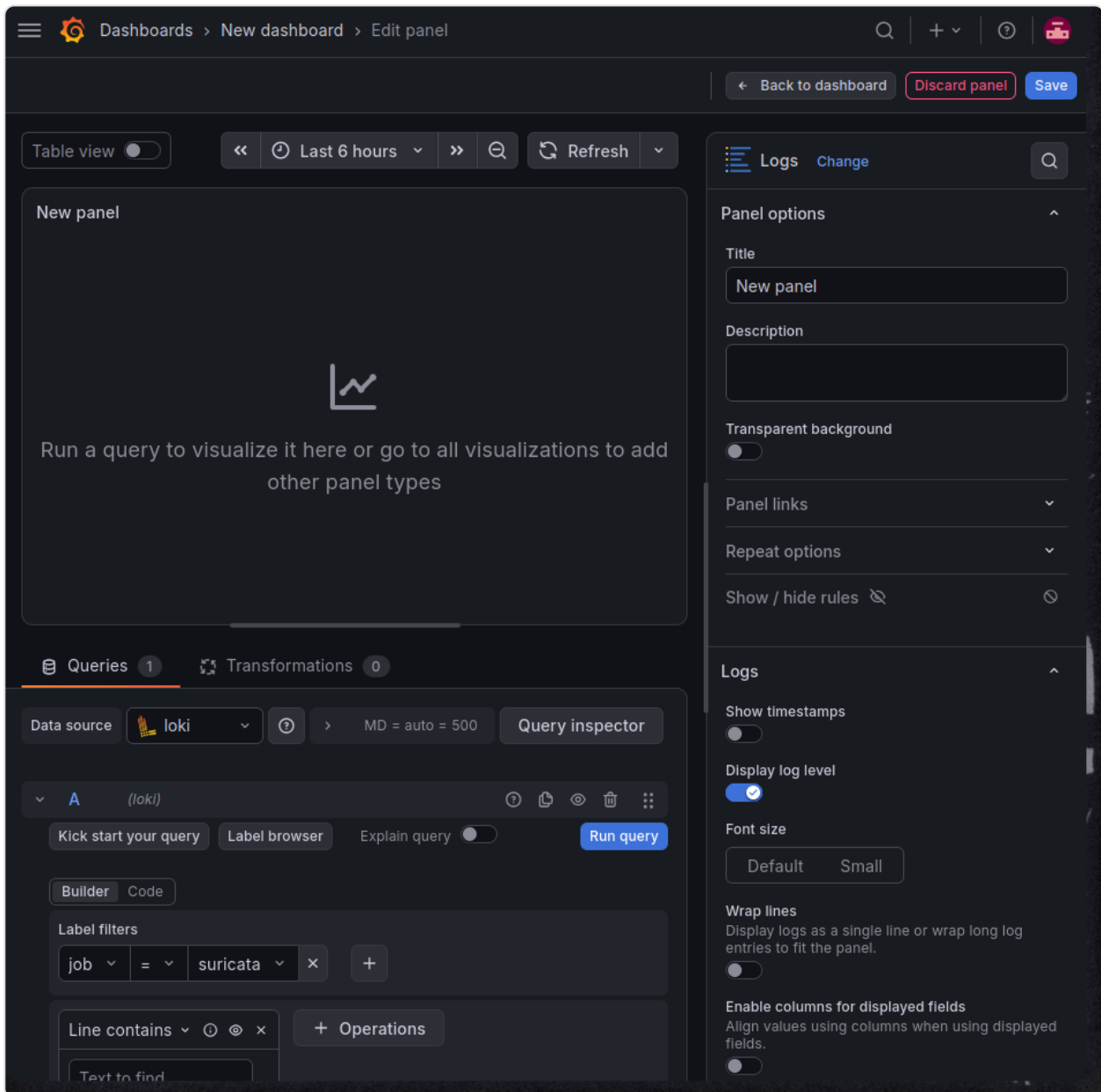
6.8 Página de inicio de Grafana



Página de bienvenida de **Grafana** tras el login. Se observa en la esquina inferior derecha del blog que la versión activa es **Grafana 13.1**, lanzada en junio de 2025. La pantalla de inicio ofrece accesos directos a:

- Documentación oficial
- Tutoriales y comunidad
- Asistente para agregar la primera fuente de datos
- Creación del primer dashboard
- Dashboards recientes y marcados como favoritos

6.9 Editor de panel — query de logs Suricata



Editor de paneles de Grafana con el tipo de visualización **Logs** y el datasource **Loki**. La query LogQL configurada usa un filtro de etiqueta:

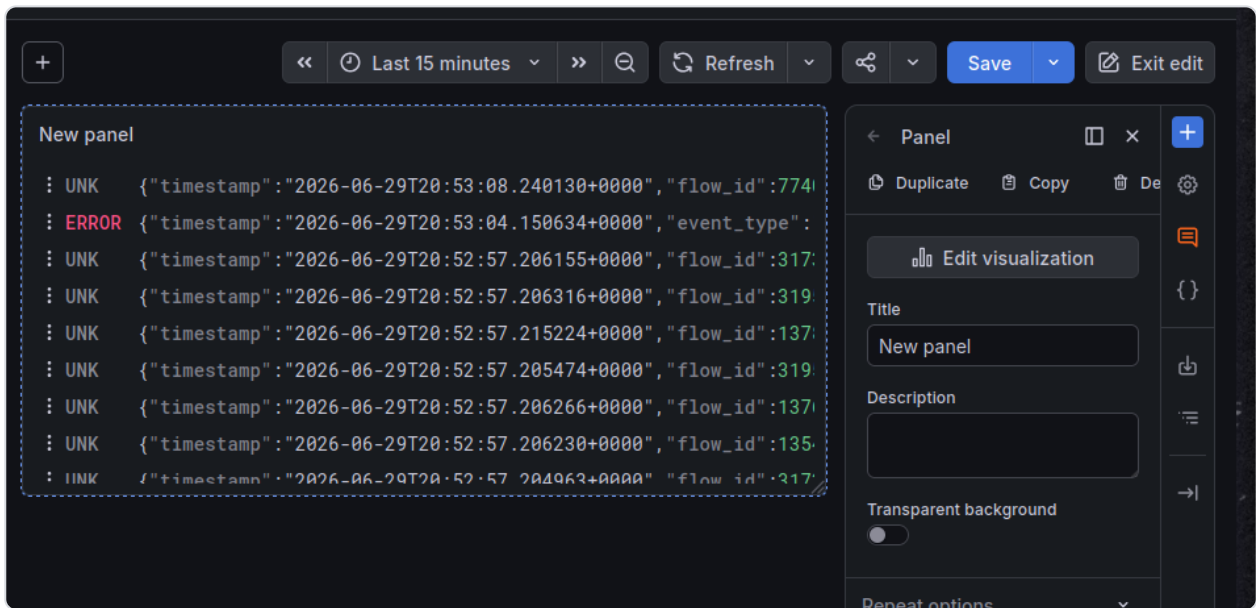
```
{job="suricata"}
```

Opciones del panel configuradas:

- **Show timestamps**: activado — muestra la marca de tiempo de cada entrada
- **Display log level**: activado — detecta automáticamente el nivel de log (ERROR, WARN, INFO)
- **Wrap lines**: activado — evita el scroll horizontal en logs largos
- **Enable columns for displayed fields**: activado

El rango de tiempo está configurado en **Last 6 hours**, mostrando los logs de `eve.json` de Suricata recibidos por Loki en las últimas 6 horas.

6.10 Panel con logs de Suricata en tiempo real



Vista del panel finalizado con **logs de Suricata en tiempo real** provenientes de `/var/log/suricata/eve.json`. Se pueden ver entradas clasificadas por nivel:

- **ERROR** (en rojo) — eventos con `event_type` crítico
- **UNK** (sin nivel reconocido) — entradas de flujo (`flow`), DNS, stats, etc.

Cada entrada muestra el timestamp y el contenido JSON completo del evento, incluyendo campos como `flow_id`, `timestamp`, `event_type`. El panel usa la vista **Last 15 minutes** y permite buscar, filtrar y explorar los logs directamente desde la interfaz.

6.11 Editor de panel — query de logs Zeek

The screenshot displays the Grafana dashboard editor interface. The main panel, titled 'zeek-visualizer', shows a list of log entries in JSON format. The entries are filtered by the query `{job="zeek"}`. The interface includes a top navigation bar with 'Table view' toggle, time range 'Last 15 minutes', and a 'Refresh' button. The right sidebar contains 'Panel options' and 'Logs' sections. The 'Panel options' section includes fields for 'Title' (zeek-visualizer), 'Description', 'Transparent background' toggle, 'Panel links', 'Repeat options', and 'Show / hide rules'. The 'Logs' section includes 'Show timestamps' toggle, 'Display log level' toggle, 'Font size' (Default/Small), 'Wrap lines' toggle, and 'Enable columns for displayed fields' toggle. The bottom section shows the 'Query inspector' with 'Data source' set to 'loki', 'MD = auto = 500', and a 'Query inspector' button. The 'Label filters' section shows a filter for 'job = zeek'. The 'Line contains' section has a 'Text to find' input field.

Panel equivalente para logs de Zeek, con la query LogQL:

```
{job="zeek"}
```

El panel muestra las entradas de `conn.log` y `dns.log` de Zeek en formato JSON, con el mismo tipo de visualización Logs. La separación por etiqueta (`job="suricata"` vs `job="zeek"`) permite crear paneles independientes para cada herramienta dentro del mismo dashboard, manteniendo la visibilidad centralizada de todo el tráfico analizado por el sensor NDR.

7. Evidencias de detección

7.1 Verificación de alertas con jq sobre eve.json

```
administrador@ndr-sensor:/opt/ndr-stack$ sudo jq -c 'select(.event_type=="alert")' /var/log/suricata/eve.json.bak | tail -5
jq: parse error: Invalid numeric literal at line 101289, column 2113
{"timestamp":"2026-06-29T21:01:51.976684+0000","flow_id":2224501528936588,"in_iface":"ens20","event_type":"alert",
src_ip":"0.0.0.0","src_port":68,"dest_ip":"255.255.255.255","dest_port":67,"proto":"UDP","ip_v":4,"pkt_src":"wire/p
cap","alert":{"action":"allowed","gid":1,"signature_id":2022973,"rev":1,"signature":"ET INFO Possible Kali Linux ho
stname in DHCP Request Packet","category":"Potential Corporate Privacy Violation","severity":1,"metadata":{"attack_
target":["Client_Endpoint"],"confidence":["Medium"],"created_at":["2016_07_18"],"deployment":["Perimeter"],"perform
ance_impact":["Low"],"reviewed_at":["2024_04_19"],"signature_severity":["Informational"],"updated_at":["2019_07_26"]
}},"app_proto":"dhcp","direction":"to_server","flow":{"pkts_toserver":1,"pkts_toclient":0,"bytes_toserver":330,"by
tes_toclient":0,"start":"2026-06-29T21:01:51.976684+0000","src_ip":"0.0.0.0","dest_ip":"255.255.255.255","src_port"
:68,"dest_port":67}}
{"timestamp":"2026-06-29T21:01:54.977844+0000","flow_id":2224501528936588,"in_iface":"ens20","event_type":"alert",
src_ip":"0.0.0.0","src_port":68,"dest_ip":"255.255.255.255","dest_port":67,"proto":"UDP","ip_v":4,"pkt_src":"wire/p
cap","alert":{"action":"allowed","gid":1,"signature_id":2022973,"rev":1,"signature":"ET INFO Possible Kali Linux ho
stname in DHCP Request Packet","category":"Potential Corporate Privacy Violation","severity":1,"metadata":{"attack_
target":["Client_Endpoint"],"confidence":["Medium"],"created_at":["2016_07_18"],"deployment":["Perimeter"],"perform
ance_impact":["Low"],"reviewed_at":["2024_04_19"],"signature_severity":["Informational"],"updated_at":["2019_07_26"]
}},"app_proto":"dhcp","direction":"to_server","flow":{"pkts_toserver":3,"pkts_toclient":0,"bytes_toserver":996,"by
tes_toclient":0,"start":"2026-06-29T21:01:51.976684+0000","src_ip":"0.0.0.0","dest_ip":"255.255.255.255","src_port"
:68,"dest_port":67}}
administrador@ndr-sensor:/opt/ndr-stack$ _
```

Consulta directa sobre el histórico de alertas almacenado en `/var/log/suricata/eve.json.bak`, filtrando únicamente los eventos de tipo `alert` con `jq`:

```
sudo jq -c 'select(.event_type=="alert")' /var/log/suricata/eve.json.bak | tail -5
```

Las últimas entradas devueltas muestran una alerta repetida:

```
{
  "event_type": "alert",
  "in_iface": "ens20",
  "proto": "UDP",
  "src_port": 68,
  "dest_port": 67,
  "alert": {
    "action": "allowed",
    "signature_id": 2022973,
    "signature": "ET INFO Possible Kali Linux hostname in DHCP Request Packet",
    "category": "Potential Corporate Privacy Violation",
    "severity": 1
  },
  "app_proto": "dhcp"
}
```

Datos clave de la evidencia:

- **Firma disparada:** `ET INFO Possible Kali Linux hostname in DHCP Request Packet` (`signature_id 2022973`) — Suricata identifica una petición DHCP (puertos 68→67) cuyo hostname del cliente delata un origen **Kali Linux**, típico de un equipo de pentesting conectado a la red monitorizada.
- **Interfaz de captura:** `ens20` , confirmando que la alerta proviene del tráfico capturado en modo promiscuo por el sensor.
- **Severidad:** 1 (informativa) y **categoría:** *Potential Corporate Privacy Violation* — el evento se clasifica como informativo y no como una amenaza crítica, pero es útil para inventariar equipos de auditoría/pentest en la red.
- El uso de `jq -c` con `select(.event_type=="alert")` permite aislar únicamente los eventos de alerta dentro del volumen de eventos que registra `eve.json` (flows, DNS, HTTP, stats, etc.), validando de forma práctica que el motor de detección de Suricata está generando alertas reales a partir del tráfico capturado.

Documentación generada el 30 de junio de 2026 basada en capturas del despliegue del laboratorio NDR sobre Proxmox VE 9.1.7.